DOCUMENTACIÓN TÉCNICA DEL PROYECTO

FEARclothes

Marketplace de ropa y sneakers: venta directa, ofertas y subastas

---

Aplicación web con Spring Boot, Thymeleaf, Bootstrap y JPA/Hibernate

Institución	**Universidad Autónoma de Chiapas**
Materia	**Programación Orientada a Objetos**
Profesor(a)	**D.s.c. Luis Gutiérrez Alfaro**
Alumno	**Bryan Gael Garcia Martinez**
Matrícula	**100051553**
Repositorio	**https://github.com/D2BLEG/fear-clothes**

Noviembre de 2026

Índice

Índice 1
1. Introducción 2
2. Objetivo General y Objetivos Específicos 3
3. Justificación 4
4. Alcance del Proyecto 5
5. Arquitectura del Sistema 7
6. Stack Tecnológico 10
7. Diseño del Sistema 11
8. Modelado de Base de Datos 14
9. Diagramas UML 19
10. Diagrama de Casos de Uso 24
11. Modelo Entidad-Relación 26
12. DBML (Database Markup Language) 28
13. Descripción de Módulos Funcionales 31
14. Estado de Desarrollo 34
15. Evidencias de Maquetación 35
16. Conclusiones 42
17. Referencias Bibliográficas 43

1. Introducción

El mercado de reventa de calzado deportivo y ropa urbana (*streetwear*) ha crecido alrededor de un modelo distinto al de una tienda tradicional: el precio no lo fija solo el vendedor, sino que surge del encuentro entre lo que pide quien vende y lo que ofrece quien compra. Plataformas como StockX popularizaron este esquema de ofertas (*bid*) y precios de venta (*ask*), además de las subastas con tiempo límite.

FEAR Clothes es una aplicación web inspirada en ese modelo. Permite que los usuarios publiquen prendas con su marca, talla, condición y fotografía, y que las vendan de tres maneras: a **precio fijo**, **aceptando ofertas** o mediante **subasta**. Los compradores pueden explorar el catálogo con filtros, comprar al instante, ofertar o pujar, y consultar su historial. Un administrador mantiene los catálogos de marcas y tallas, modera publicaciones y gestiona cuentas.

El sistema se desarrolló como proyecto de Programación Orientada a Objetos. Su construcción busca demostrar, sobre un problema realista, el uso de abstracción, herencia, polimorfismo y encapsulamiento, junto con patrones de diseño (Strategy, Factory, Template Method y Repository), la arquitectura Modelo-Vista-Controlador y la persistencia relacional con JPA/Hibernate.

Este documento recopila el análisis, el diseño, el modelado de datos, los diagramas UML, la arquitectura implementada y el estado de avance del proyecto.

2. Objetivo General y Objetivos Específicos

2.1 Objetivo general

Diseñar, desarrollar e implementar una aplicación web funcional de compraventa de ropa y calzado, denominada **FEAR Clothes**, que permita vender prendas por precio fijo, por ofertas o por subasta, aplicando los principios de la Programación Orientada a Objetos, el patrón arquitectónico MVC y una base de datos relacional gestionada con JPA/Hibernate.

2.2 Objetivos específicos

1. Levantar los requisitos funcionales y no funcionales del sistema y representarlos en un diagrama de casos de uso.
2. Modelar las clases del dominio aplicando abstracción, herencia, polimorfismo y encapsulamiento, e identificar los patrones de diseño que resuelven los problemas centrales (tipos de venta y creación de prendas).
3. Diseñar una base de datos relacional normalizada, con más de seis tablas vinculadas mediante llaves primarias y foráneas, y documentarla con un diagrama entidad-relación, un script SQL y su definición en DBML.
4. Implementar el backend con Spring Boot siguiendo una arquitectura MVC por capas (controlador, servicio, repositorio y entidad).
5. Construir la interfaz de usuario con HTML, CSS, Bootstrap y Thymeleaf, de forma responsiva y coherente con la identidad visual de la marca.
6. Incorporar autenticación y autorización por roles (comprador, vendedor y administrador) con Spring Security.
7. Implementar operaciones CRUD completas sobre las entidades principales, con validación de datos y reglas de negocio.
8. Documentar el sistema y planificar su despliegue en la nube de AWS.

3. Justificación

Relevancia del dominio. Un marketplace de compraventa con tres modalidades de precio es un problema con reglas de negocio ricas: cada modalidad valida las ofertas de forma distinta, calcula su precio vigente de otra manera y se cierra bajo condiciones diferentes. Esa variación de comportamiento es justamente el escenario donde el polimorfismo y el patrón Strategy aportan más valor que una cadena de condicionales.

Valor académico. El proyecto integra en una sola aplicación los temas de la materia: jerarquías de herencia reales (usuarios y prendas), interfaces y clases abstractas, encapsulamiento de reglas dentro de las entidades, fábricas para crear objetos y una separación estricta por capas. Además, obliga a resolver el paso del modelo de objetos al modelo relacional (mapeo objeto-relacional) y a justificar decisiones de normalización.

Valor técnico. Se emplean herramientas de uso profesional: Spring Boot, Spring Data JPA, Spring Security, Thymeleaf y Bootstrap. El código no depende de una base de datos concreta (H2 en desarrollo y MySQL en la nube) ni de un medio de almacenamiento específico para las imágenes, lo que facilita el despliegue posterior en AWS.

Valor práctico. La aplicación es un prototipo funcional de una tienda real de ropa urbana: se puede demostrar el flujo completo de publicar, ofertar, comprar y administrar, con datos de prueba incluidos.

4. Alcance del Proyecto

4.1 Alcance

El sistema incluye los siguientes módulos, todos implementados y funcionales en el entorno local:

- Registro e inicio de sesión de usuarios con contraseñas cifradas, tres roles (Comprador, Vendedor y Administrador) y edición del perfil.
- Catálogo público con búsqueda por texto y filtros por categoría, marca, talla y tipo de venta.
- Publicación de prendas (calzado, camisetas y chamarras) con fotografía, y su edición y eliminación.
- Tres tipos de venta: precio fijo, precio de venta con ofertas y subasta con fecha de cierre.
- Compra inmediata con pago simulado y registro de órdenes.
- Cierre automático de subastas vencidas mediante una tarea programada.
- Panel de administración: marcas, tallas, usuarios y moderación de publicaciones.

4.2 Fuera de alcance

Se excluyen de esta etapa los pagos reales, el cálculo y la gestión de envíos, la verificación de autenticidad de las prendas, el chat entre usuarios, las notificaciones por correo y la aplicación móvil. Se mencionan como trabajo futuro.

4.3 Requisitos funcionales

ID	Requisito
RF-01	El sistema permitirá registrar usuarios como comprador o vendedor, con correo único y contraseña cifrada.
RF-02	El sistema permitirá iniciar y cerrar sesión, y restringirá las funciones según el rol.
RF-03	Cualquier visitante podrá consultar el catálogo y el detalle de una prenda.
RF-04	El catálogo permitirá buscar por texto y filtrar por categoría, marca, talla y tipo de venta.
RF-05	Un vendedor podrá publicar una prenda indicando categoría, marca, talla, condición, color, descripción, fotografía, tipo de venta y precio.
RF-06	Un vendedor podrá editar sus publicaciones activas y eliminarlas mientras no tengan ofertas ni compras.
RF-07	Un usuario autenticado podrá comprar al instante una publicación de precio fijo o con ofertas.
RF-08	Un usuario autenticado podrá dejar una oferta en una publicación con ofertas y el vendedor podrá aceptarla.
RF-09	Un usuario autenticado podrá pujar en una subasta; cada puja deberá superar a la anterior por el incremento mínimo.
RF-10	Al vencer una subasta, el sistema asignará la venta al mejor postor o la cerrará si no hubo pujas.
RF-11	El usuario podrá consultar sus compras; el vendedor, sus publicaciones y ventas.
RF-12	El administrador podrá crear, consultar, editar y eliminar marcas y tallas.
RF-13	El administrador podrá consultar usuarios, activarlos o desactivarlos, y cerrar publicaciones.

ID	Requisito
RF-14	Todo usuario autenticado podrá editar su nombre y cambiar su contraseña desde «Mi perfil».

4.4 Requisitos no funcionales

ID	Requisito
RNF-01	Arquitectura MVC por capas y código orientado a objetos con bajo acoplamiento.
RNF-02	Las contraseñas se almacenan cifradas con BCrypt; las rutas se protegen con Spring Security y tokens CSRF.
RNF-03	Interfaz responsiva basada en Bootstrap 5, utilizable en computadora y teléfono.
RNF-04	Persistencia con JPA/Hibernate sobre una base relacional normalizada (H2 local y MySQL en AWS).
RNF-05	Validación de datos de entrada en servidor con Bean Validation y mensajes claros al usuario.
RNF-06	El almacenamiento de imágenes debe poder cambiarse (disco local o S3) sin modificar la lógica de negocio.
RNF-07	El sistema debe poder ejecutarse localmente con un solo comando y datos de prueba precargados.

5. Arquitectura del Sistema

FEAR Clothes sigue el patrón **Modelo-Vista-Controlador (MVC)** sobre una arquitectura por capas (figura al final de esta sección). Cada petición HTTP atraviesa el filtro de seguridad, llega a un controlador, que delega la lógica en un servicio; el servicio trabaja con las entidades del dominio y persiste mediante repositorios; finalmente el controlador elige la vista que se renderiza con Thymeleaf.

5.1 Correspondencia con MVC

Componente MVC	Responsabilidad	Implementación
Modelo	Entidades del dominio, reglas de negocio y estrategias de venta.	Paquetes `model` y `venta`; servicios en `service`.
Vista	Presentación HTML con plantillas Thymeleaf y estilos Bootstrap.	`src/main/resources/templates` y `static`.
Controlador	Recibe peticiones, valida formularios, invoca servicios y elige la vista.	Paquete `controller`.

5.2 Capas

Capa	Paquete	Función
Presentación	`controller`, plantillas	Rutas HTTP, formularios, mensajes al usuario.
Seguridad	`security`	Autenticación por correo, roles y protección de rutas y CSRF.
Negocio	`service`, `venta`, `factory`	Reglas de ofertas, subastas, compras y administración; transacciones.
Persistencia	`repository`	Interfaces Spring Data JPA y filtros dinámicos (Specifications).
Dominio	`model`	Entidades JPA con su comportamiento.
Infraestructura	`storage`, `config`	Almacenamiento de imágenes, datos de prueba y configuración web.

5.3 Flujo de una petición

Como ejemplo, al pulsar «Comprar ahora» el navegador envía `POST /publicacion/{id}/comprar`. Spring Security verifica la sesión y el token CSRF; `PublicacionController` obtiene al usuario autenticado y llama a `PublicacionService.comprarAhora()`. El servicio abre una transacción, carga la publicación, pide a su estrategia de venta que valide la compra, crea la `Orden`, marca la publicación como vendida y confirma la transacción. El controlador redirige a «Mis compras», cuya vista se genera con Thymeleaf.

5.4 Arquitectura de despliegue

En desarrollo la aplicación se ejecuta como un proceso Java con servidor Tomcat embebido y una base H2 almacenada en archivo. Para producción se propone: una instancia **EC2** (o Elastic Beanstalk) que ejecuta el `.jar`

con el perfil `aws`, una base **RDS MySQL** y un bucket **S3** para las imágenes. Las credenciales se entregan por variables de entorno.

5.5 Diagrama de arquitectura

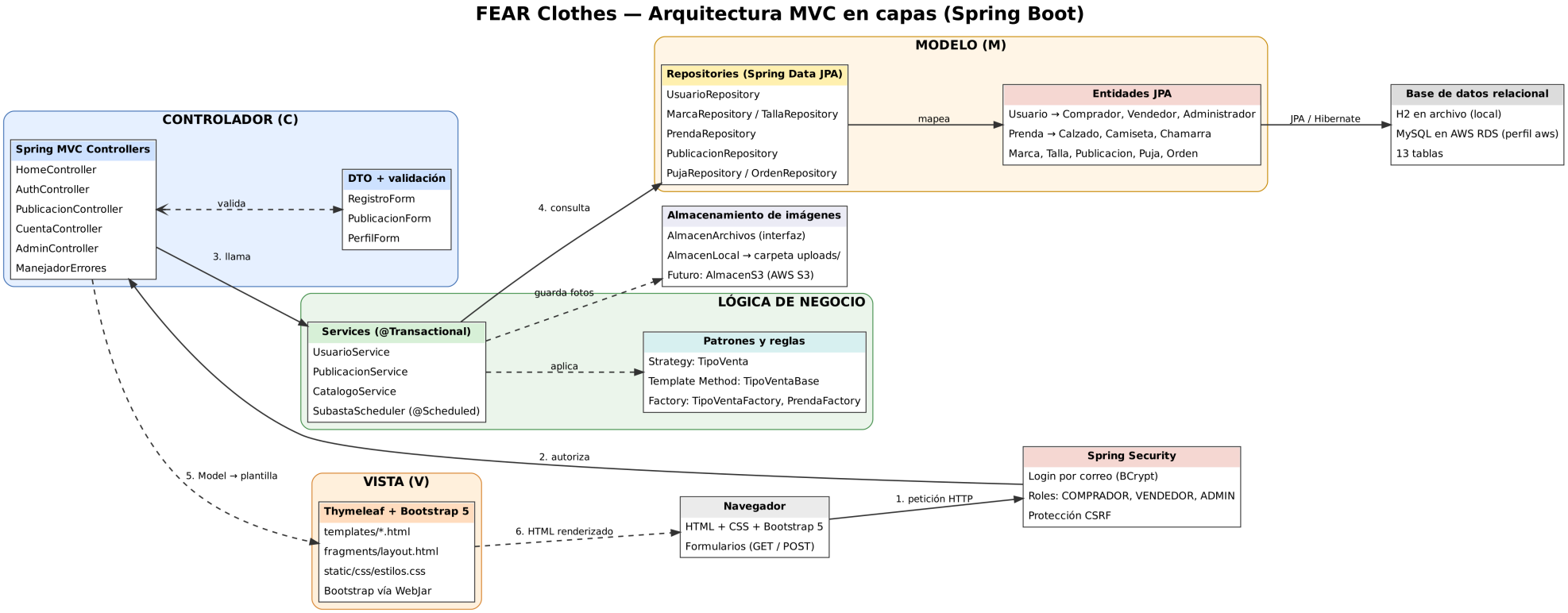


Figura 1. Arquitectura MVC por capas de FEAR Clothes.

6. Stack Tecnológico

Capa	Tecnología	Uso en el proyecto
Lenguaje	Java 21	Lenguaje principal; programación orientada a objetos.
Framework backend	Spring Boot 3.3	Configuración automática, servidor embebido y ejecución de la aplicación.
Web / MVC	Spring MVC	Controladores, enlace de formularios y manejo de errores.
Persistencia	Spring Data JPA + Hibernate	Mapeo objeto-relacional, repositorios y consultas dinámicas.
Seguridad	Spring Security 6	Inicio de sesión, roles, BCrypt y protección CSRF.
Validación	Jakarta Bean Validation	Reglas de entrada en los formularios (DTO).
Motor de plantillas	Thymeleaf + extras Spring Security	Vistas dinámicas y contenido según el rol.
Interfaz	HTML5, CSS3, Bootstrap 5.3	Diseño responsivo; Bootstrap se empaqueta como WebJar para funcionar sin internet.
Base de datos local	H2 (modo archivo)	Desarrollo y demostración; los datos persisten entre ejecuciones.
Base de datos en la nube	MySQL 8 (Amazon RDS)	Producción; se activa con el perfil `aws`.
Construcción	Maven	Gestión de dependencias y empaquetado del `.jar`.
Entorno de desarrollo	IntelliJ IDEA	IDE del equipo.
Control de versiones	Git y GitHub	Repositorio del código fuente.
Nube	AWS (EC2, RDS, S3)	Despliegue planificado (siguiente etapa).
Modelado	Mermaid, Graphviz, DBML	Diagramas UML y entidad-relación; definición de la base en dbdiagram.io.

7. Diseño del Sistema

7.1 Principios de la programación orientada a objetos

Principio	Dónde se aplica
Abstracción	`Usuario` y `Prenda` son clases abstractas que capturan lo común; `TipoVenta` es una interfaz que define el contrato de una forma de vender.
Herencia	`Usuario` → `Comprador`, `Vendedor`, `Administrador`; `Prenda` → `Calzado`, `Camiseta`, `Chamarra`; `TipoVentaBase` → `VentaDirecta`, `VentaPorOferta`, `Subasta`.
Polimorfismo	`getRol()` y `puedePublicar()` varían por tipo de usuario; `getDetalleEspecifico()` por tipo de prenda; `Publicacion` y `PublicacionService` llaman a `TipoVenta` sin conocer la implementación concreta.
Encapsulamiento	Atributos privados con acceso por métodos; las reglas de ofertas viven dentro de cada estrategia y no en los controladores.
Composición	`Publicacion` agrupa una `Prenda`, un vendedor, una lista de `Puja` y delega en su `TipoVenta`.

7.2 Patrones de diseño

Patrón	Clases	Problema que resuelve
Strategy	`TipoVenta`, `VentaDirecta`, `VentaPorOferta`, `Subasta`	Cada tipo de venta valida ofertas, calcula el precio y decide su cierre de forma distinta; se evita una cadena de `if` / `switch` en los servicios.
Template Method	`TipoVentaBase`	`validarPuja()` fija los pasos comunes (publicación activa, no ser el vendedor, monto válido) y delega el último paso a `validarMonto()` de cada subclase.
Factory	`TipoVentaFactory`, `PrendaFactory`	Entregan la estrategia correcta según el tipo de publicación y crean la subclase de `Prenda` según la categoría elegida en el formulario.
Repository	Paquete `repository`	Aísla el acceso a datos detrás de interfaces Spring Data JPA.
DTO / Form	`PublicacionForm`, `RegistroForm`	Separan los datos del formulario de las entidades y concentran la validación de entrada.
Inyección de dependencias	Constructores de servicios y controladores	Bajo acoplamiento; las dependencias (por ejemplo `AlmacenArchivos`) se resuelven por interfaz.

7.3 Reglas de los tipos de venta

Tipo	Precio vigente	¿Compra inmediata?	Regla de oferta	Cierre
Venta directa	Precio fijo	Sí	No admite ofertas	Al comprarse

Tipo	Precio vigente	¿Compra inmediata?	Regla de oferta	Cierre
Con ofertas (bid/ask)	Precio de venta del vendedor	Sí	La oferta debe ser menor al precio de venta; el vendedor decide si la acepta	Al comprarse o al aceptar una oferta
Subasta	Puja más alta (o precio inicial)	No	Cada puja supera a la anterior al menos en $50 y debe hacerse antes del cierre	Automático en la fecha de cierre; gana la puja más alta

7.4 Seguridad y control de acceso

Función	Visitante	Comprador	Vendedor	Administrador
Ver catálogo y detalle	Sí	Sí	Sí	Sí
Registrarse / iniciar sesión	Sí	—	—	—
Comprar, ofertar y pujar	No	Sí	Sí (no en lo propio)	Sí
Publicar, editar y eliminar prendas propias	No	No	Sí	No
Aceptar ofertas	No	No	Sí (sus publicaciones)	No
Mis compras	No	Sí	Sí	Sí
Mis ventas	No	No	Sí	No
Marcas, tallas, usuarios y moderación	No	No	No	Sí

Las contraseñas se cifran con BCrypt. Todos los formularios incluyen token CSRF. Un usuario desactivado por el administrador no puede iniciar sesión.

7.5 Estructura del proyecto

```
fear-clothes/
├── pom.xml
├── README.md
├── database/
│   └── fear_clothes.sql          Script de creación de la base de datos (MySQL 8)
├── docs/
│   ├── fear_clothes.dbml         Definición del esquema en DBML
│   └── diagrams/                 Diagramas ER, UML, casos de uso y arquitectura
└── src/main/
    ├── java/com/fearclothes/
    │   ├── FearClothesApplication.java
    │   ├── model/        Entidades: Usuario, Comprador, Vendedor, Administrador,
    │   │                 Prenda, Calzado, Camiseta, Chamarra, Marca, Talla,
    │   │                 Publicacion, Puja, Orden y enumeraciones
    │   ├── venta/        Strategy: TipoVenta, TipoVentaBase, VentaDirecta,
    │   │                 VentaPorOferta, Subasta, TipoVentaFactory
    │   ├── factory/      PrendaFactory
    │   ├── repository/   Repositorios Spring Data JPA y filtros (Specifications)
    │   ├── service/      UsuarioService, CatalogoService, PublicacionService,
    │   │                 SubastaScheduler
    │   ├── controller/   Home, Auth, Publicacion, Cuenta, Admin, ManejadorErrores
    │   ├── dto/          RegistroForm, PublicacionForm (validación)
    │   ├── security/     SecurityConfig, UsuarioDetailsService
    │   ├── storage/      AlmacenArchivos (interfaz), AlmacenLocal
    │   ├── config/       DataLoader (datos de prueba), WebConfig
    │   └── exception/    ReglaNegocioException
```

```
    └── resources/
        ├── templates/    Vistas Thymeleaf (Bootstrap 5)
        ├── static/       Estilos propios e imágenes de ejemplo
        └── application*.properties   Configuración (local y perfil aws)
```

8. Modelado de Base de Datos

La base de datos contiene **13 tablas** relacionadas mediante llaves primarias y foráneas, por encima del mínimo de seis solicitado. Se diseñó a partir del modelo de objetos y se verificó contra las formas normales.

8.1 Tablas

Grupo	Tablas	Descripción
Usuarios	`usuarios`, `compradores`, `vendedores`, `administradores`	Datos comunes en `usuarios`; cada rol especializa en su propia tabla.
Catálogos	`marcas`, `tallas`	Datos maestros reutilizados por todas las prendas.
Prendas	`prendas`, `calzados`, `camisetas`, `chamarras`	Datos comunes en `prendas`; atributos propios en cada subtipo.
Operación	`publicaciones`, `pujas`, `ordenes`	Venta de una prenda, ofertas recibidas y compras concretadas.

8.2 Normalización

- **Primera forma normal (1FN).** Todos los atributos son atómicos y no existen grupos repetidos: las ofertas de una publicación están en `pujas` (una fila por oferta) y no en una lista dentro de `publicaciones`.
- **Segunda forma normal (2FN).** Todas las tablas usan una llave primaria simple (`id`), por lo que ningún atributo puede depender solo de una parte de la llave.
- **Tercera forma normal (3FN).** No hay dependencias transitivas: el nombre y país de la marca viven en `marcas` y no se repiten en cada prenda; la etiqueta de talla vive en `tallas`; los datos del vendedor en `vendedores`; los atributos exclusivos de cada tipo de prenda en su subtabla, sin columnas vacías.
- **Integridad.** Las relaciones se aseguran con llaves foráneas; `marcas.nombre`, `tallas.etiqueta` y `usuarios.email` son únicos; `ordenes.publicacion_id` es único para impedir vender dos veces la misma publicación.

Decisión de diseño. Para la herencia se eligió la estrategia *tabla por subclase* (`JOINED` en JPA) en lugar de una tabla única. Una tabla única mezclaría en `prendas` columnas como `silueta`, `tipo_cuello` e `impermeable`, que serían nulas en dos de cada tres filas. Con una tabla por subclase cada dato vive donde corresponde, a cambio de un *join* adicional al consultar. La columna `categoria` de `prendas` identifica el subtipo y permite filtrar el catálogo sin unir las subtablas.

`ordenes.precio_final` se guarda aunque podría deducirse de la publicación o de la puja ganadora, porque es el precio histórico pactado y no debe cambiar si la publicación se edita después.

8.3 Diccionario de datos

PK = llave primaria · FK = llave foránea · UK = valor único.

`usuarios` Datos comunes de todas las cuentas.

Columna	Tipo	Clave	Descripción
`id`	BIGINT	PK	Identificador autoincremental.
`nombre`	VARCHAR(255)		Nombre del usuario.
`email`	VARCHAR(255)	UK	Correo; se usa para iniciar sesión.
`password`	VARCHAR(255)		Contraseña cifrada con BCrypt.
`activo`	BOOLEAN		Indica si la cuenta puede iniciar sesión.

`compradores` Especialización de usuarios con rol Comprador.

Columna	Tipo	Clave	Descripción
`id`	BIGINT	PK, FK	Referencia a usuarios.id.

`vendedores` Especialización de usuarios con rol Vendedor.

Columna	Tipo	Clave	Descripción
`id`	BIGINT	PK, FK	Referencia a usuarios.id.
`nombre_tienda`	VARCHAR(255)		Nombre comercial del vendedor.

`administradores` Especialización de usuarios con rol Administrador.

Columna	Tipo	Clave	Descripción
`id`	BIGINT	PK, FK	Referencia a usuarios.id.

`marcas` Catálogo de marcas.

Columna	Tipo	Clave	Descripción
`id`	BIGINT	PK	Identificador.
`nombre`	VARCHAR(255)	UK	Nombre de la marca.
`pais`	VARCHAR(255)		País de origen.

`tallas` Catálogo de tallas de ropa y calzado.

Columna	Tipo	Clave	Descripción
`id`	BIGINT	PK	Identificador.
`etiqueta`	VARCHAR(255)	UK	Texto de la talla (S, M, 8 MX…).

`prendas` Datos comunes de toda prenda.

Columna	Tipo	Clave	Descripción
`id`	BIGINT	PK	Identificador.
`nombre`	VARCHAR(255)		Nombre de la prenda.
`descripcion`	VARCHAR(1000)		Descripción libre.
`color`	VARCHAR(255)		Color principal.
`imagen_url`	VARCHAR(255)		Ruta o URL de la fotografía.
`categoria`	ENUM		CALZADO, CAMISETA o CHAMARRA; identifica el subtipo.
`condicion`	ENUM		NUEVA, COMO_NUEVA o USADA.
`marca_id`	BIGINT	FK	Referencia a marcas.id.
`talla_id`	BIGINT	FK	Referencia a tallas.id.

`calzados` Atributos exclusivos del calzado.

Columna	Tipo	Clave	Descripción
`id`	BIGINT	PK, FK	Referencia a prendas.id.
`silueta`	VARCHAR(255)		Silueta (alta, baja…).

`camisetas` Atributos exclusivos de las camisetas.

Columna	Tipo	Clave	Descripción
`id`	BIGINT	PK, FK	Referencia a prendas.id.
`tipo_cuello`	VARCHAR(255)		Tipo de cuello.

`chamarras` Atributos exclusivos de las chamarras.

Columna	Tipo	Clave	Descripción
`id`	BIGINT	PK, FK	Referencia a prendas.id.
`impermeable`	BOOLEAN		Indica si es impermeable.

`publicaciones` Una prenda puesta a la venta.

Columna	Tipo	Clave	Descripción
`id`	BIGINT	PK	Identificador.
`prenda_id`	BIGINT	FK, UK	Prenda publicada (una publicación por prenda).
`vendedor_id`	BIGINT	FK	Usuario que vende.
`tipo`	ENUM		DIRECTA, OFERTA o SUBASTA.
`precio`	DECIMAL(12,2)		Precio fijo, precio de venta o precio inicial.
`fecha_cierre`	DATETIME		Cierre de la subasta (nulo en los demás tipos).
`estado`	ENUM		ACTIVA, VENDIDA o CERRADA.
`fecha_creacion`	DATETIME		Fecha de alta.

`pujas` Ofertas y pujas recibidas.

Columna	Tipo	Clave	Descripción
`id`	BIGINT	PK	Identificador.
`publicacion_id`	BIGINT	FK	Publicación ofertada.
`postor_id`	BIGINT	FK	Usuario que oferta.
`monto`	DECIMAL(12,2)		Monto ofertado.
`fecha`	DATETIME		Fecha y hora de la oferta.

`ordenes` Compras concretadas (pago simulado).

Columna	Tipo	Clave	Descripción
`id`	BIGINT	PK	Identificador.
`comprador_id`	BIGINT	FK	Usuario que compra.
`publicacion_id`	BIGINT	FK, UK	Publicación vendida.
`precio_final`	DECIMAL(12,2)		Precio pactado.
`metodo_pago`	VARCHAR(255)		Método de pago (simulado).
`fecha`	DATETIME		Fecha de la compra.

8.4 Script de creación

El script completo está en `database/fear_clothes.sql` (MySQL 8). Crea la base de datos, las 13 tablas con sus llaves primarias, foráneas, restricciones `UNIQUE` y `CHECK`, índices, y carga los catálogos iniciales. Fragmento:

```
CREATE TABLE prendas (
    id          BIGINT        NOT NULL AUTO_INCREMENT,
    nombre      VARCHAR(255)  NOT NULL,
    descripcion VARCHAR(1000) NULL,
    color       VARCHAR(255)  NULL,
    imagen_url  VARCHAR(255)  NULL COMMENT 'Ruta /img/..., /uploads/... o URL externa',
    categoria   ENUM('CALZADO','CAMISETA','CHAMARRA')   NOT NULL,
    condicion   ENUM('NUEVA','COMO_NUEVA','USADA')      NOT NULL,
    marca_id    BIGINT        NOT NULL,
    talla_id    BIGINT        NOT NULL,
    CONSTRAINT pk_prendas PRIMARY KEY (id),
    CONSTRAINT fk_prendas_marca FOREIGN KEY (marca_id)
        REFERENCES marcas (id) ON DELETE RESTRICT,
    CONSTRAINT fk_prendas_talla FOREIGN KEY (talla_id)
        REFERENCES tallas (id) ON DELETE RESTRICT
) ENGINE=InnoDB;

CREATE TABLE publicaciones (
    id             BIGINT        NOT NULL AUTO_INCREMENT,
    prenda_id      BIGINT        NOT NULL,
    vendedor_id    BIGINT        NOT NULL,
    tipo           ENUM('DIRECTA','OFERTA','SUBASTA') NOT NULL,
    precio         DECIMAL(12,2) NOT NULL,
    fecha_cierre   DATETIME(6)   NULL,
    estado         ENUM('ACTIVA','VENDIDA','CERRADA') NOT NULL DEFAULT 'ACTIVA',
    fecha_creacion DATETIME(6)   NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    CONSTRAINT pk_publicaciones PRIMARY KEY (id),
    CONSTRAINT uq_publicaciones_prenda UNIQUE (prenda_id),
    CONSTRAINT fk_publicaciones_prenda FOREIGN KEY (prenda_id)
        REFERENCES prendas (id) ON DELETE RESTRICT,
    CONSTRAINT fk_publicaciones_vendedor FOREIGN KEY (vendedor_id)
        REFERENCES usuarios (id) ON DELETE RESTRICT,
```

```
    CONSTRAINT ck_publicaciones_precio CHECK (precio > 0),
    -- Una subasta siempre debe tener fecha de cierre
    CONSTRAINT ck_publicaciones_subasta CHECK (tipo <> 'SUBASTA' OR fecha_cierre IS NOT NULL)
) ENGINE=InnoDB;

CREATE TABLE pujas (
    id             BIGINT        NOT NULL AUTO_INCREMENT,
    publicacion_id BIGINT        NOT NULL,
    postor_id      BIGINT        NOT NULL,
    monto          DECIMAL(12,2) NOT NULL,
    fecha          DATETIME(6)   NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    CONSTRAINT pk_pujas PRIMARY KEY (id),
    CONSTRAINT fk_pujas_publicacion FOREIGN KEY (publicacion_id)
        REFERENCES publicaciones (id) ON DELETE CASCADE,
    CONSTRAINT fk_pujas_postor FOREIGN KEY (postor_id)
        REFERENCES usuarios (id) ON DELETE RESTRICT,
    CONSTRAINT ck_pujas_monto CHECK (monto > 0)
) ENGINE=InnoDB;
```

9. Diagramas UML

9.1 Diagrama de clases del dominio

Muestra las jerarquías de herencia (`Usuario` y `Prenda`), la composición de `Publicacion` con su prenda, sus pujas y su orden, y las enumeraciones del modelo.

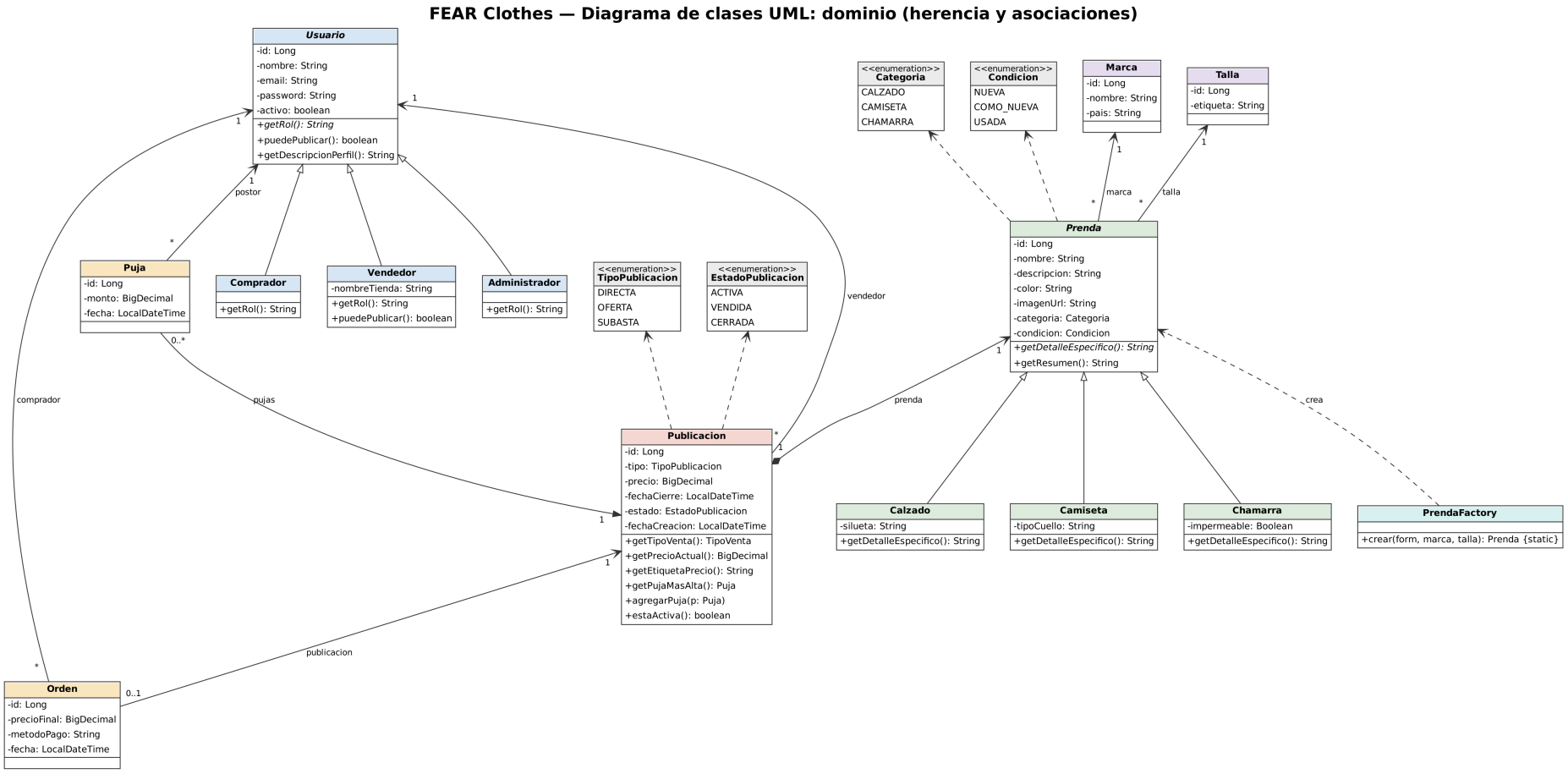


Figura 2. Diagrama de clases UML del dominio.

9.2 Patrón Strategy y fábricas

Detalla la interfaz `TipoVenta`, la clase abstracta `TipoVentaBase` (Template Method), las tres estrategias concretas y las fábricas `TipoVentaFactory` y `PrendaFactory`. El diagrama de clases completo, con todo en una sola imagen, está en `docs/diagrams/uml_clases.png`.

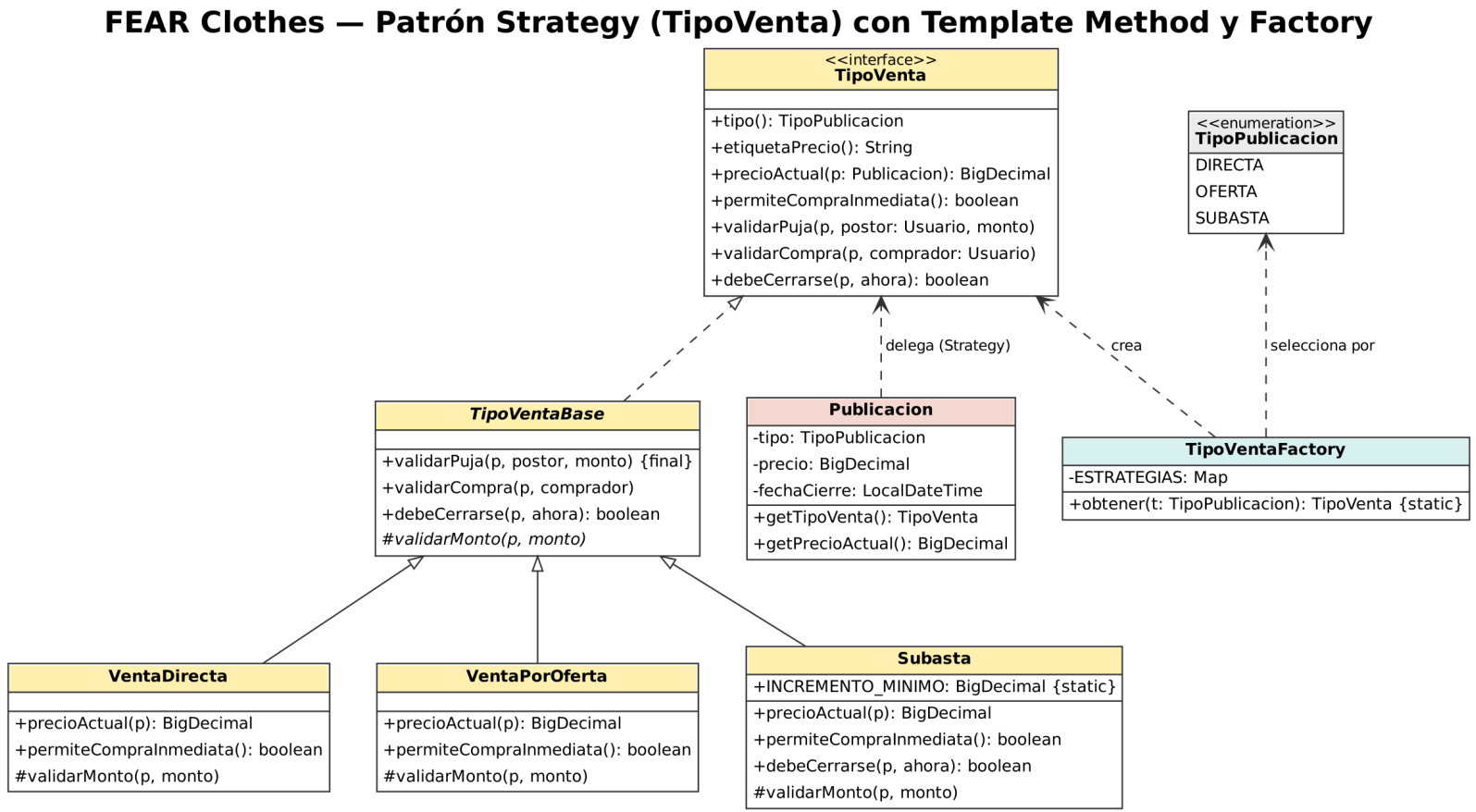


Figura 3. Patrón Strategy de los tipos de venta y fábricas.

9.3 Diagrama de clases por capas

Relaciona controladores, servicios, repositorios y entidades, así como la abstracción de almacenamiento de imágenes.

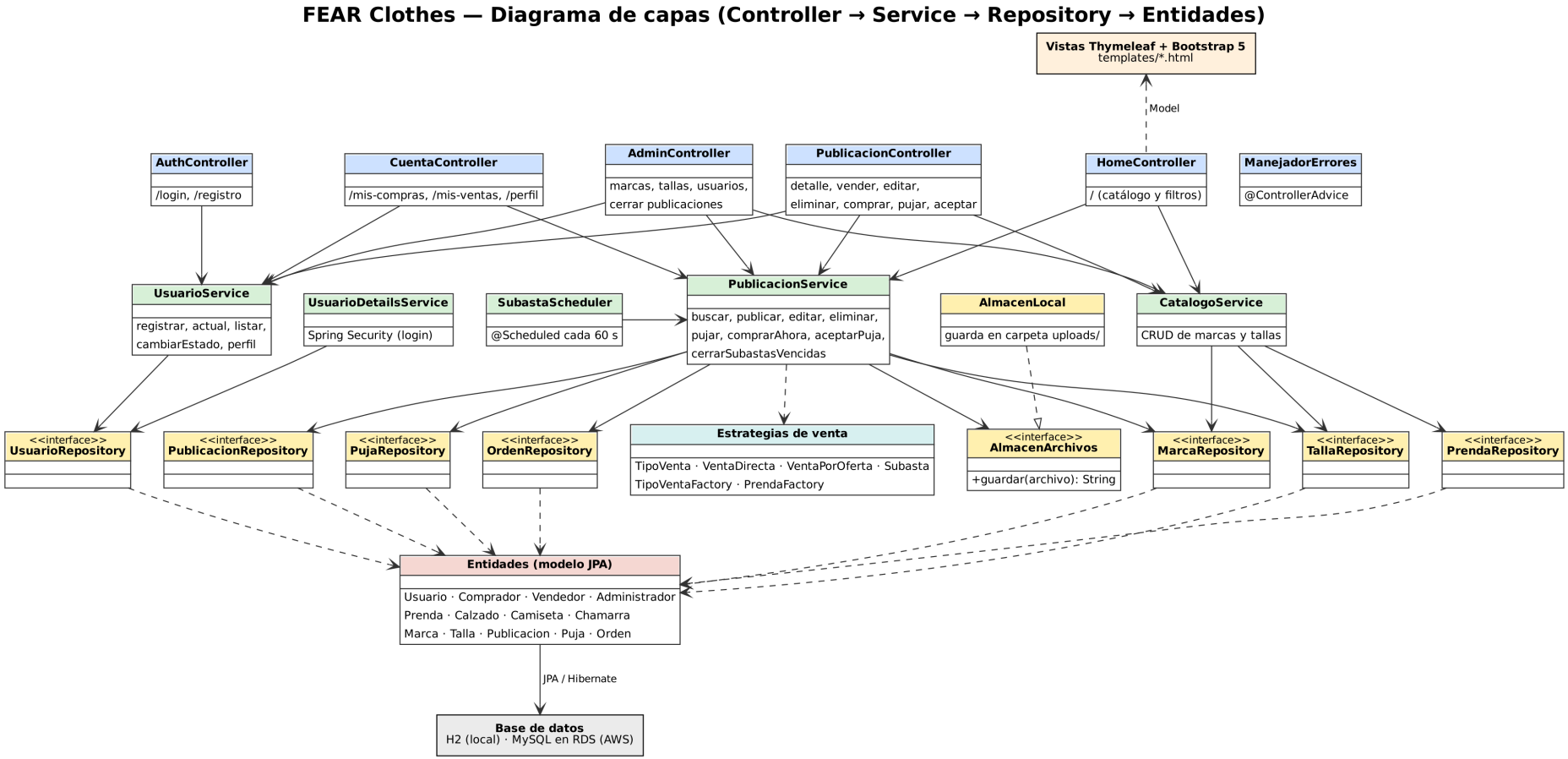


Figura 4. Diagrama de clases por capas (Controller, Service, Repository).

9.4 Secuencia de una puja y cierre de subasta

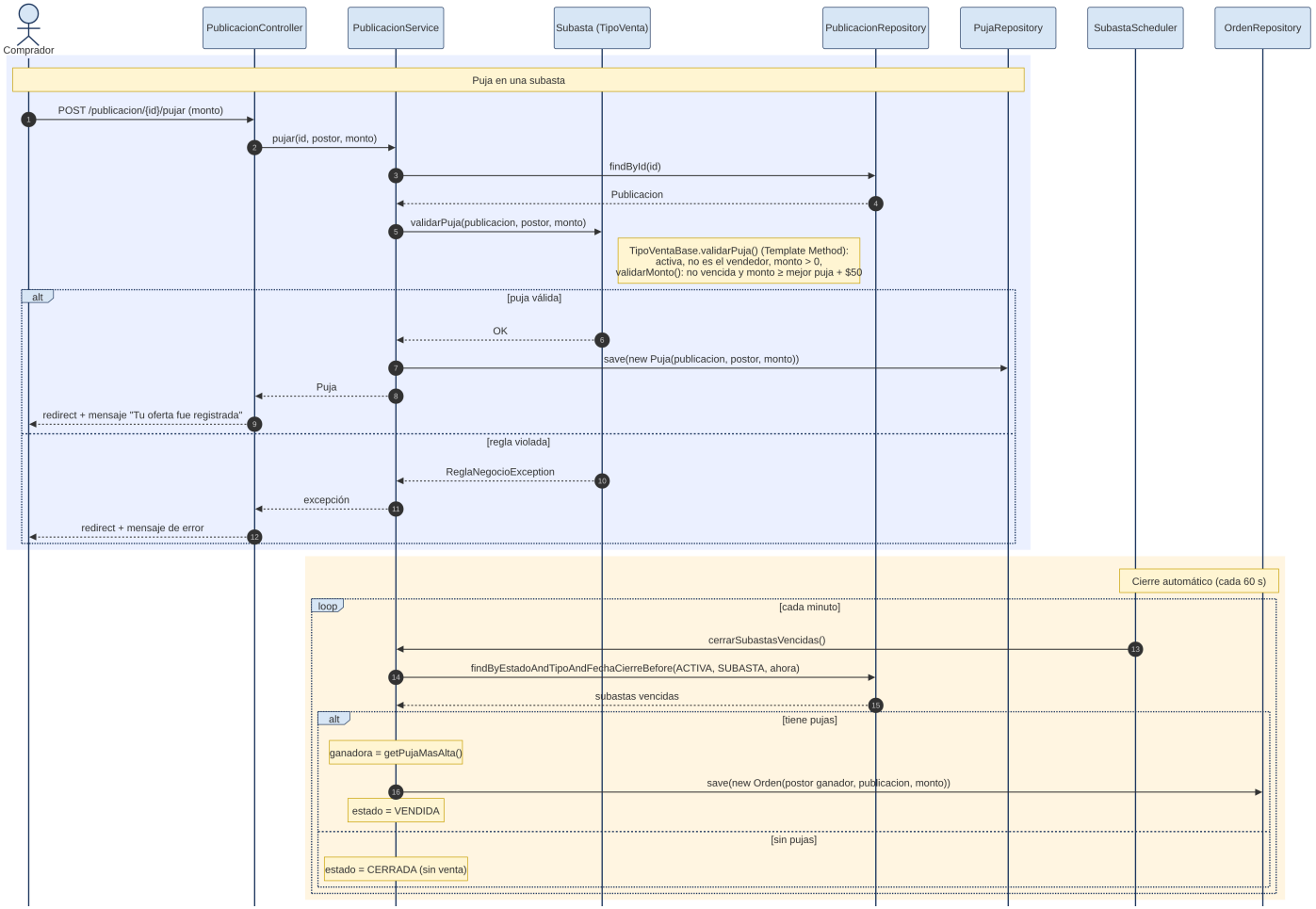


Figura 1. Diagrama de secuencia: puja en una subasta y cierre automático.

9.5 Secuencia de una compra inmediata

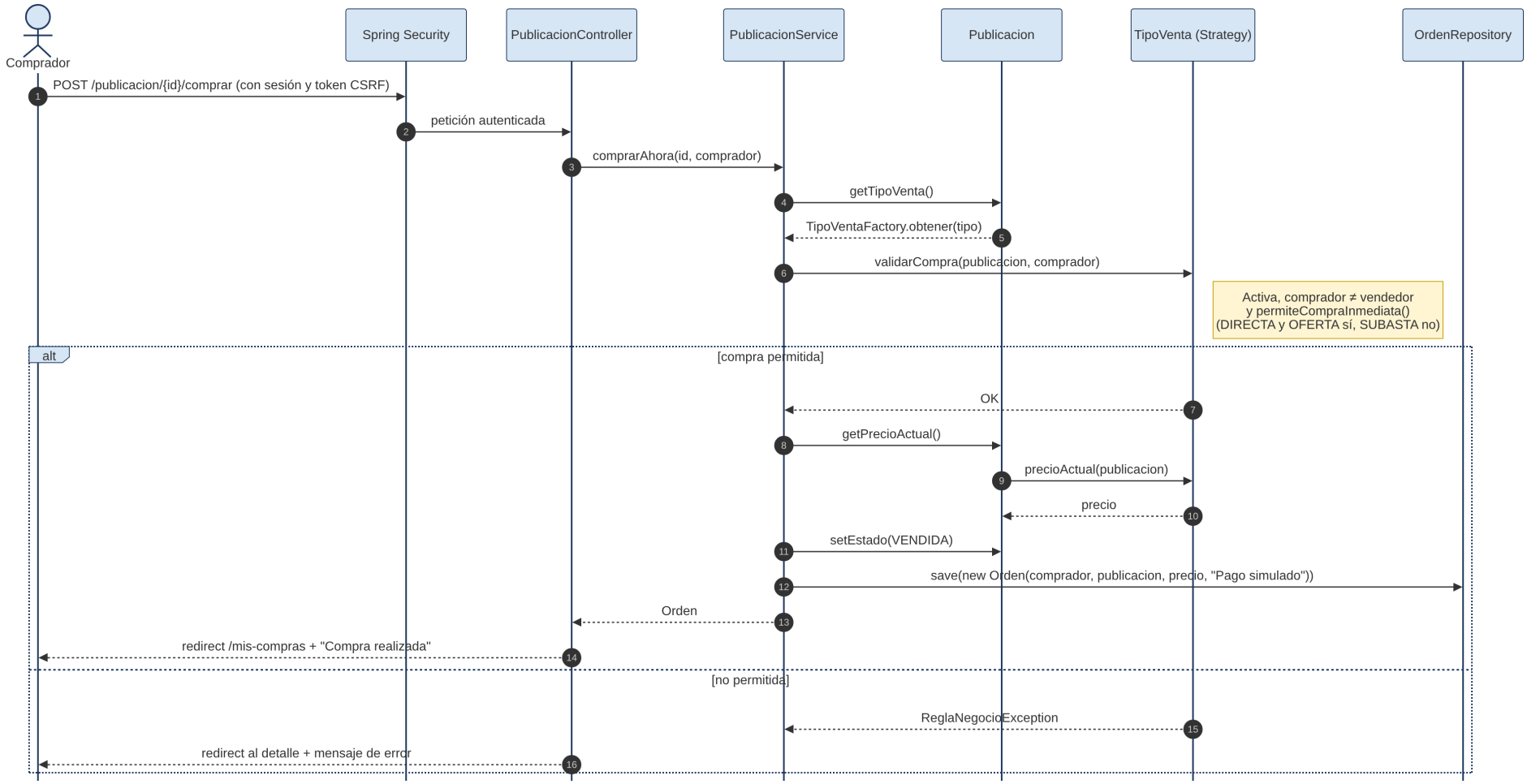


Figura 2. Diagrama de secuencia: compra inmediata de una publicación.

10. Diagrama de Casos de Uso

Identifica cuatro actores: **Visitante** (sin sesión), **Comprador**, **Vendedor** y **Administrador**, además del **Temporizador** que cierra las subastas vencidas.

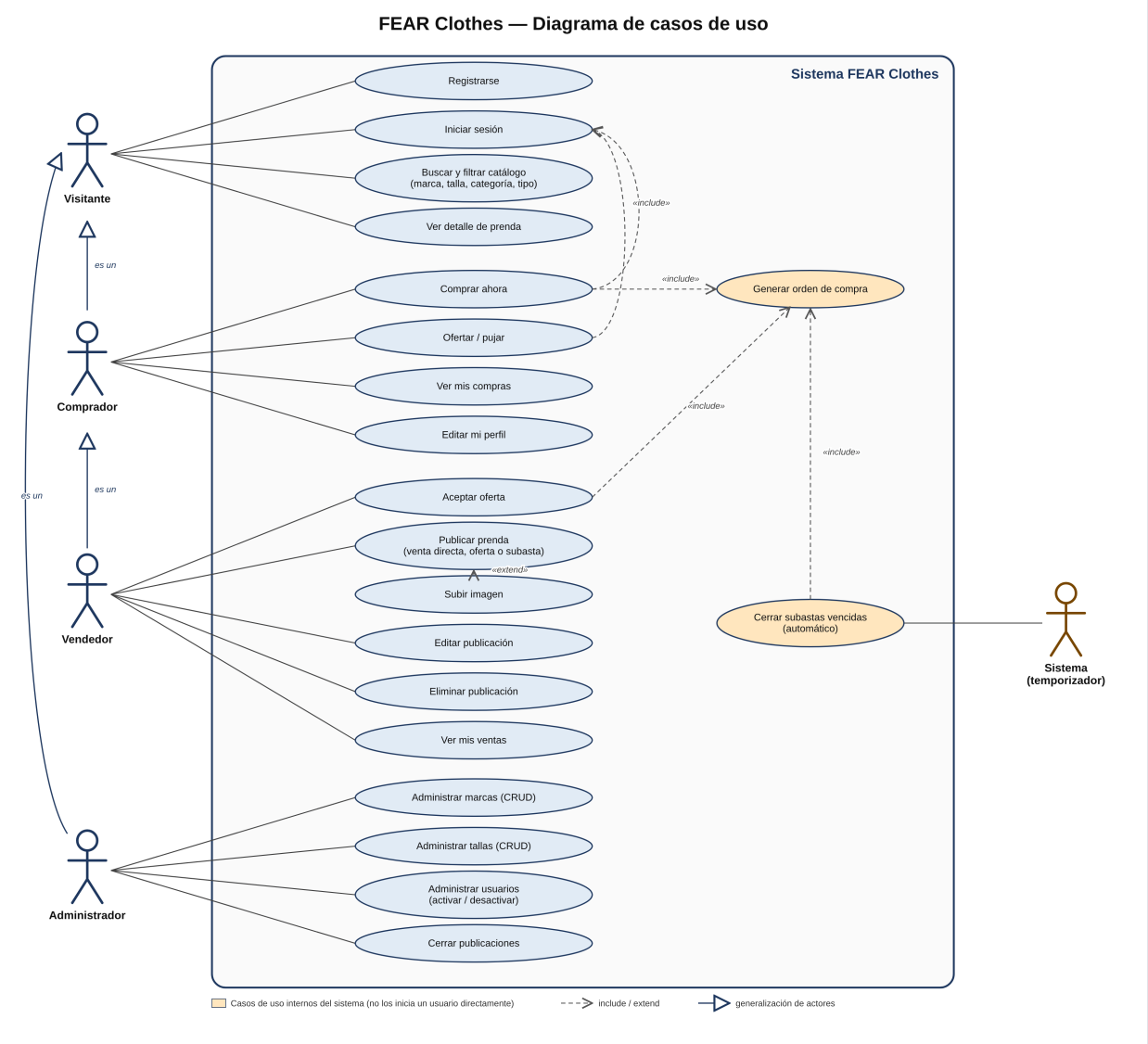


Figura 5. Diagrama de casos de uso de FEAR Clothes.

10.1 Descripción de los casos de uso principales

Caso de uso	Actor	Descripción	Resultado
CU-01 Registrarse	Visitante	Captura nombre, correo, contraseña y rol (comprar, o comprar y vender).	Cuenta creada con contraseña cifrada.
CU-02 Iniciar sesión	Visitante	Ingresa correo y contraseña.	Sesión iniciada; el menú muestra las opciones del rol.
CU-03 Explorar catálogo	Todos	Busca por texto y filtra por categoría, marca, talla y tipo de venta.	Lista de publicaciones activas.

Caso de uso	Actor	Descripción	Resultado
CU-04 Publicar prenda	Vendedor	Captura los datos de la prenda, sube una foto y elige tipo de venta y precio.	Publicación activa en el catálogo.
CU-05 Editar / eliminar publicación	Vendedor	Modifica los datos de una publicación activa o la elimina si no tiene ofertas.	Publicación actualizada o eliminada.
CU-06 Comprar ahora	Comprador, Vendedor	Compra al precio publicado (venta directa u oferta).	Orden creada; publicación marcada como vendida.
CU-07 Ofertar o pujar	Comprador, Vendedor	Registra una oferta o puja que cumple las reglas del tipo de venta.	Puja guardada; el precio de una subasta se actualiza.
CU-08 Aceptar oferta	Vendedor	Elige una de las ofertas recibidas de su publicación.	Orden creada al precio de la oferta.
CU-09 Cerrar subastas vencidas	Temporizador	Cada minuto revisa las subastas cuya fecha de cierre pasó.	Venta al mejor postor o cierre sin venta.
CU-10 Consultar compras y ventas	Comprador, Vendedor	Revisa su historial de órdenes y publicaciones.	Listados personales.
CU-11 Administrar marcas y tallas	Administrador	Crea, edita y elimina elementos de los catálogos.	Catálogos actualizados.
CU-12 Administrar usuarios	Administrador	Consulta usuarios y activa o desactiva cuentas.	Estado de cuenta actualizado.
CU-13 Moderar publicaciones	Administrador	Cierra publicaciones activas.	Publicación cerrada.
CU-14 Editar mi perfil	Todo usuario autenticado	Cambia su nombre y, opcionalmente, su contraseña.	Perfil actualizado.

11. Modelo Entidad-Relación

El modelo representa las 13 entidades con sus atributos clave y las cardinalidades de cada relación.

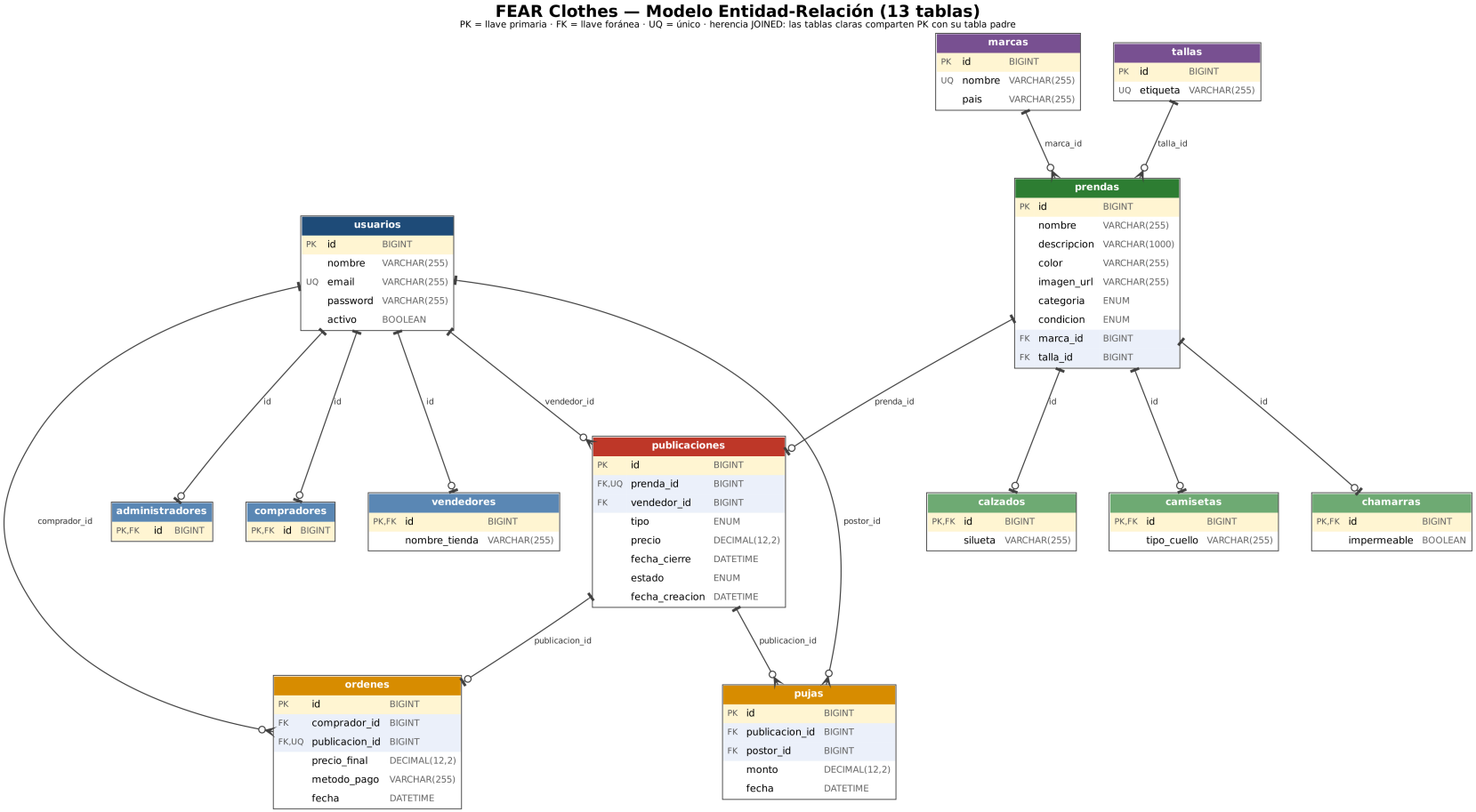


Figura 6. Modelo entidad-relación de FEAR Clothes (13 tablas).

11.1 Relaciones

Relación	Cardinalidad	Llave foránea
Usuario — Comprador / Vendedor / Administrador	1 a 0..1 (especialización)	`compradores.id`, `vendedores.id`, `administradores.id` → `usuarios.id`
Prenda — Calzado / Camiseta / Chamarra	1 a 0..1 (especialización)	`calzados.id`, `camisetas.id`, `chamarras.id` → `prendas.id`
Marca — Prenda	1 a N	`prendas.marca_id` → `marcas.id`
Talla — Prenda	1 a N	`prendas.talla_id` → `tallas.id`
Prenda — Publicación	1 a 1	`publicaciones.prenda_id` → `prendas.id` (único)
Usuario (vendedor) — Publicación	1 a N	`publicaciones.vendedor_id` → `usuarios.id`
Publicación — Puja	1 a N	`pujas.publicacion_id` → `publicaciones.id`
Usuario (postor) — Puja	1 a N	`pujas.postor_id` → `usuarios.id`
Usuario (comprador) — Orden	1 a N	`ordenes.comprador_id` → `usuarios.id`
Publicación — Orden	1 a 0..1	`ordenes.publicacion_id` → `publicaciones.id` (único)

12. DBML (Database Markup Language)

DBML es un lenguaje declarativo para describir esquemas de bases de datos. La definición completa está en `docs/fear_clothes.dbml`; puede pegarse en *dbdiagram.io* para visualizar el diagrama y exportar el SQL.

```
// FEAR Clothes - Modelo de base de datos en DBML (https://dbdiagram.io)
// Pegar este contenido en dbdiagram.io para ver el diagrama.
// Herencia JPA (JOINED): usuarios -> compradores/vendedores/administradores
//                        prendas  -> calzados/camisetas/chamarras

Project fear_clothes {
  database_type: 'MySQL'
  Note: 'Marketplace de ropa y sneakers: venta directa, ofertas (bid/ask) y subastas'
}

Enum categoria_prenda {
  CALZADO
  CAMISETA
  CHAMARRA
}

Enum condicion_prenda {
  NUEVA
  COMO_NUEVA
  USADA
}

Enum tipo_publicacion {
  DIRECTA [note: 'Precio fijo']
  OFERTA  [note: 'Precio de venta (ask) + ofertas (bids)']
  SUBASTA [note: 'Gana la puja más alta al cierre']
}

Enum estado_publicacion {
  ACTIVA
  VENDIDA
  CERRADA
}

// ------------------------------------------------------------- usuarios
Table usuarios {
  id bigint [pk, increment]
  nombre varchar(255) [not null]
  email varchar(255) [not null, unique]
  password varchar(255) [not null, note: 'Hash BCrypt']
  activo boolean [not null, default: true]

  Note: 'Superclase abstracta Usuario'
}

Table compradores {
  id bigint [pk, note: 'PK y FK a usuarios.id']
  Note: 'Subclase Comprador'
}

Table vendedores {
  id bigint [pk, note: 'PK y FK a usuarios.id']
  nombre_tienda varchar(255)
  Note: 'Subclase Vendedor'
}

Table administradores {
  id bigint [pk, note: 'PK y FK a usuarios.id']
  Note: 'Subclase Administrador'
}

// ------------------------------------------------------------- catálogos
Table marcas {
  id bigint [pk, increment]
  nombre varchar(255) [not null, unique]
  pais varchar(255)
```

```
}

Table tallas {
  id bigint [pk, increment]
  etiqueta varchar(255) [not null, unique, note: 'S, M, L... u 8 MX, 9 MX...']
}

// ------------------------------------------------------------ prendas
Table prendas {
  id bigint [pk, increment]
  nombre varchar(255) [not null]
  descripcion varchar(1000)
  color varchar(255)
  imagen_url varchar(255) [note: 'Ruta /img, /uploads o URL externa']
  categoria categoria_prenda [not null]
  condicion condicion_prenda [not null]
  marca_id bigint [not null]
  talla_id bigint [not null]

  Note: 'Superclase abstracta Prenda'
}

Table calzados {
  id bigint [pk, note: 'PK y FK a prendas.id']
  silueta varchar(255)
}

Table camisetas {
  id bigint [pk, note: 'PK y FK a prendas.id']
  tipo_cuello varchar(255)
}

Table chamarras {
  id bigint [pk, note: 'PK y FK a prendas.id']
  impermeable boolean
}

// ------------------------------------------------------------ ventas
Table publicaciones {
  id bigint [pk, increment]
  prenda_id bigint [not null, unique]
  vendedor_id bigint [not null]
  tipo tipo_publicacion [not null]
  precio decimal(12,2) [not null, note: 'Precio fijo, ask o precio inicial de subasta']
  fecha_cierre datetime [note: 'Solo subastas']
  estado estado_publicacion [not null, default: 'ACTIVA']
  fecha_creacion datetime [not null]

  indexes {
    (estado, tipo) [name: 'ix_publicaciones_estado_tipo']
  }
}

Table pujas {
  id bigint [pk, increment]
  publicacion_id bigint [not null]
  postor_id bigint [not null]
  monto decimal(12,2) [not null]
  fecha datetime [not null]

  indexes {
    (publicacion_id, monto) [name: 'ix_pujas_publicacion_monto']
  }
}

Table ordenes {
  id bigint [pk, increment]
  comprador_id bigint [not null]
  publicacion_id bigint [not null, unique]
  precio_final decimal(12,2) [not null]
  metodo_pago varchar(255) [note: 'Pago simulado']
  fecha datetime [not null]
}

// ------------------------------------------------------------ relaciones
// Herencia (1 a 0..1)
```

```
Ref: compradores.id - usuarios.id [delete: cascade]
Ref: vendedores.id - usuarios.id [delete: cascade]
Ref: administradores.id - usuarios.id [delete: cascade]
Ref: calzados.id - prendas.id [delete: cascade]
Ref: camisetas.id - prendas.id [delete: cascade]
Ref: chamarras.id - prendas.id [delete: cascade]

// Prenda
Ref: prendas.marca_id > marcas.id [delete: restrict]
Ref: prendas.talla_id > tallas.id [delete: restrict]

// Publicaciones
Ref: publicaciones.prenda_id - prendas.id [delete: restrict]
Ref: publicaciones.vendedor_id > usuarios.id [delete: restrict]

// Pujas y órdenes
Ref: pujas.publicacion_id > publicaciones.id [delete: cascade]
Ref: pujas.postor_id > usuarios.id [delete: restrict]
Ref: ordenes.comprador_id > usuarios.id [delete: restrict]
Ref: ordenes.publicacion_id - publicaciones.id [delete: restrict]
```

13. Descripción de Módulos Funcionales

13.1 Autenticación y usuarios

Permite crear una cuenta como comprador o como vendedor e iniciar sesión con correo y contraseña. Las contraseñas se cifran con BCrypt y la sesión determina qué opciones ve el usuario. Cada usuario puede editar su nombre y contraseña, y el administrador puede activar o desactivar cuentas.

Aspecto	Detalle
Actores	Visitante, Comprador, Vendedor, Administrador
Clases principales	`Usuario` y subclases, `UsuarioService`, `UsuarioDetailsService`, `SecurityConfig`, `RegistroForm`, `PerfilForm`
Rutas	`GET/POST /registro`, `GET /login`, `POST /logout`, `GET/POST /perfil`, `GET /admin/usuarios`, `POST /admin/usuarios/{id}/estado`
Vistas	`registro`, `login`, `perfil`, `admin-usuarios`

13.2 Catálogo y búsqueda

Página principal con todas las publicaciones activas. Se puede buscar por texto y filtrar por categoría, marca, talla y tipo de venta. Los filtros se combinan dinámicamente con *Specifications* de Spring Data.

Aspecto	Detalle
Actores	Todos
Clases principales	`HomeController`, `PublicacionService.buscar()`, `PublicacionSpecs`
Rutas	`GET /`, `GET /publicacion/{id}`
Vistas	`index`, `detalle`

13.3 Publicaciones (CRUD)

El vendedor crea publicaciones eligiendo categoría, marca, talla, condición, tipo de venta y precio; puede editarlas mientras estén activas y eliminarlas si no tienen ofertas ni compras. `PrendaFactory` crea la subclase de `Prenda` que corresponde a la categoría.

Aspecto	Detalle
Actores	Vendedor (propias), Administrador
Clases principales	`Publicacion`, `Prenda` y subclases, `PrendaFactory`, `PublicacionForm`, `PublicacionService`
Rutas	`GET/POST /vender`, `GET/POST /publicacion/{id}/editar`, `POST /publicacion/{id}/eliminar`, `POST /publicacion/{id}/cerrar`
Vistas	`vender`, `editar-publicacion`, `detalle`, `mis-ventas`

13.4 Ofertas y subastas

Cada publicación delega sus reglas en una estrategia `TipoVenta`. En venta con ofertas el comprador puede pagar el precio o dejar una oferta menor que el vendedor puede aceptar; en subasta cada puja debe superar a la anterior por $50 como mínimo. Una tarea programada cierra cada minuto las subastas vencidas y asigna la venta al mejor postor.

Aspecto	Detalle
Actores	Comprador, Vendedor, Temporizador
Clases principales	`TipoVenta`, `TipoVentaBase`, `VentaDirecta`, `VentaPorOferta`, `Subasta`, `TipoVentaFactory`, `Puja`, `SubastaScheduler`
Rutas	`POST /publicacion/{id}/pujar`, `POST /publicacion/{id}/aceptar/{pujaId}`
Vistas	`detalle`

13.5 Compras y órdenes

La compra inmediata genera una `Orden` con el precio pactado y marca la publicación como vendida, dentro de una transacción. El pago es simulado. El usuario consulta su historial de compras y el vendedor sus ventas.

Aspecto	Detalle
Actores	Comprador, Vendedor
Clases principales	`Orden`, `PublicacionService.comprarAhora()`, `CuentaController`
Rutas	`POST /publicacion/{id}/comprar`, `GET /mis-compras`, `GET /mis-ventas`
Vistas	`mis-compras`, `mis-ventas`

13.6 Administración

Panel exclusivo del administrador para mantener los catálogos de marcas y tallas (alta, consulta, edición y baja), revisar las publicaciones activas y cerrarlas, y gestionar el estado de las cuentas. Una marca o talla en uso no puede eliminarse.

Aspecto	Detalle
Actores	Administrador
Clases principales	`CatalogoService`, `AdminController`, `Marca`, `Talla`
Rutas	`GET /admin`, `/admin/marcas…`, `/admin/tallas…`, `POST /admin/publicaciones/{id}/cerrar`
Vistas	`admin`, `admin-marca-form`, `admin-talla-form`, `admin-usuarios`

13.7 Gestión de imágenes

El vendedor sube una fotografía (JPG, PNG o WEBP de hasta 5 MB) con vista previa; el archivo se guarda con un nombre generado y se sirve bajo `/uploads`. La interfaz `AlmacenArchivos` permite sustituir el disco local por Amazon S3 sin cambiar el resto del código.

Aspecto	Detalle
Actores	Vendedor
Clases principales	`AlmacenArchivos`, `AlmacenLocal`, `WebConfig`, `ManejadorErrores`
Rutas	`GET /uploads/**`
Vistas	`vender`

14. Estado de Desarrollo

14.1 Avance por módulo

Módulo	Estado	Observaciones
Autenticación y usuarios	Implementado	Registro, inicio de sesión, roles, BCrypt, activación de cuentas.
Catálogo y búsqueda	Implementado	Filtros dinámicos por texto, categoría, marca, talla y tipo de venta.
Publicaciones (CRUD)	Implementado	Alta, consulta, edición y eliminación con reglas de propiedad.
Ofertas y subastas	Implementado	Strategy con tres tipos de venta; cierre automático programado.
Compras y órdenes	Implementado	Pago simulado; historial de compras y ventas.
Administración	Implementado	Marcas y tallas (CRUD), usuarios, moderación de publicaciones.
Imágenes	Implementado	Subida local con vista previa; interfaz lista para S3.
Interfaz Bootstrap	Implementado	Plantillas Thymeleaf con Bootstrap 5, tema oscuro de la marca.
Base de datos y scripts	Implementado	13 tablas, script MySQL, DBML y datos de prueba.
Despliegue en AWS	Pendiente	Planificado: EC2 o Elastic Beanstalk, RDS MySQL y S3.
Pruebas automatizadas	Pendiente	Pruebas unitarias de las estrategias de venta y de los servicios.
Pagos y envíos reales	Fuera de alcance	Trabajo futuro.

14.2 Plan de despliegue en AWS

1. Generar el ejecutable con `mvn clean package`.
2. Crear una base de datos Amazon RDS MySQL y ejecutar `database/fear_clothes.sql`.
3. Crear una instancia EC2 con Java 21 y copiar el `.jar`.
4. Definir las variables de entorno `DB_HOST`, `DB_USER` y `DB_PASSWORD`.
5. Ejecutar `java -jar fear-clothes.jar --spring.profiles.active=aws` y abrir el puerto en el grupo de seguridad.
6. Crear un bucket S3 e implementar `AlmacenArchivos` con el SDK de AWS para guardar las imágenes fuera de la instancia.

14.3 Trabajo futuro

Integrar una pasarela de pago real, notificaciones por correo, calificación de vendedores, verificación de autenticidad, paginación del catálogo y pruebas automatizadas con cobertura de las reglas de negocio.

15. Evidencias de Maquetación

Esta sección reúne las pantallas de la aplicación en funcionamiento. Todas las vistas se construyen con plantillas Thymeleaf y componentes de Bootstrap 5, con un tema oscuro y el rojo de la marca como color de acento.

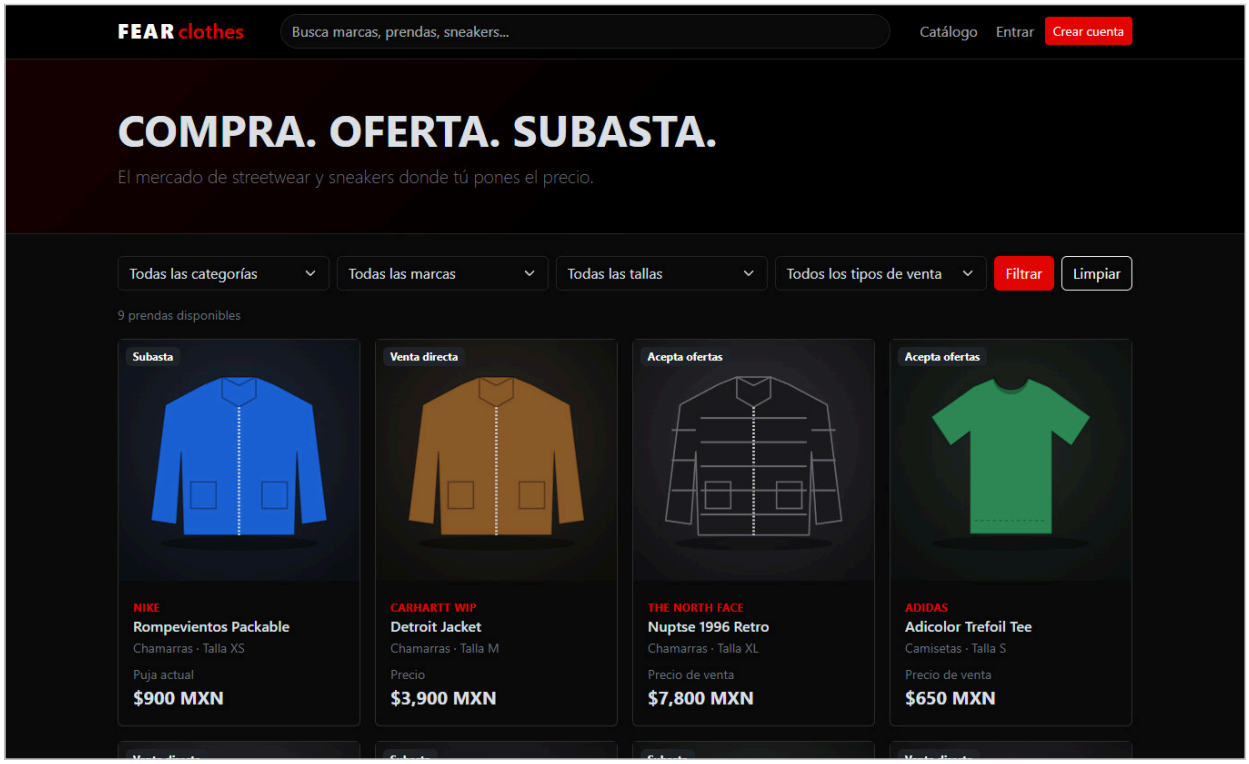


Figura 7. Catálogo con filtros. Página principal: hero, filtros y tarjetas de prendas.

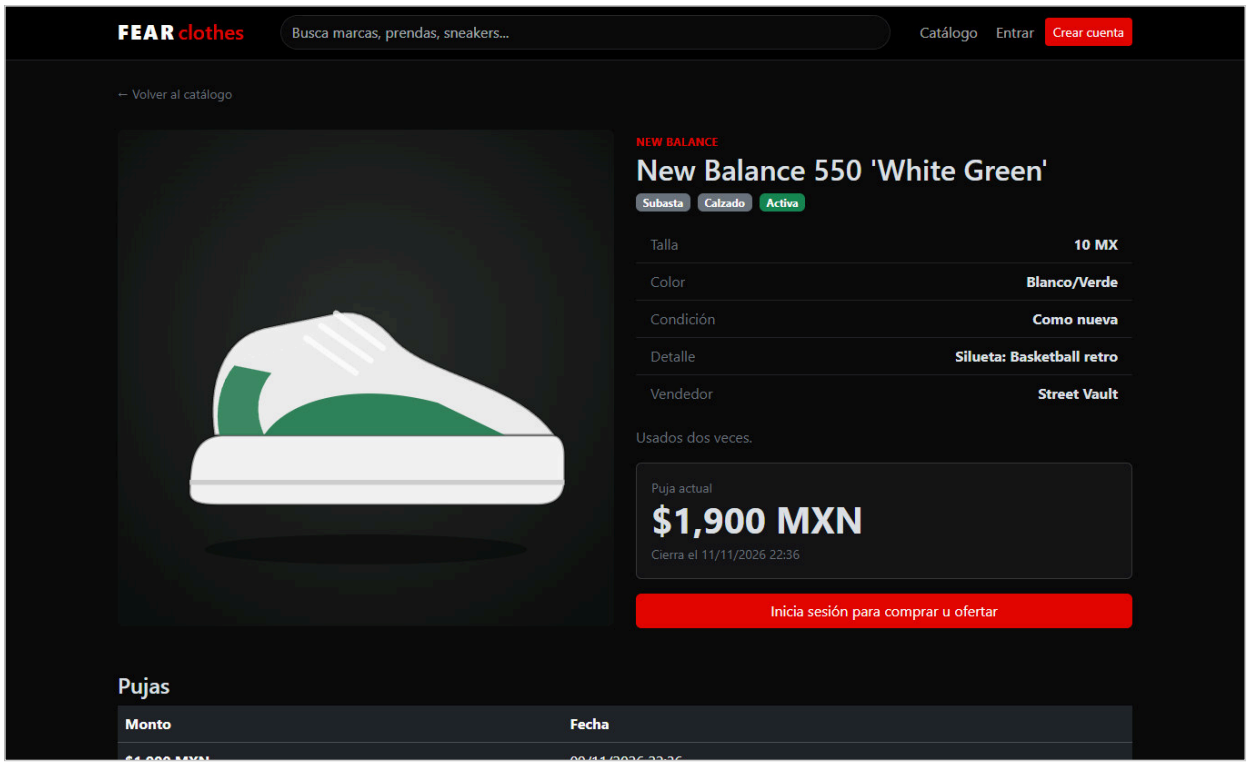


Figura 8. Detalle de una subasta. Pujas recibidas, fecha de cierre y formulario de puja.

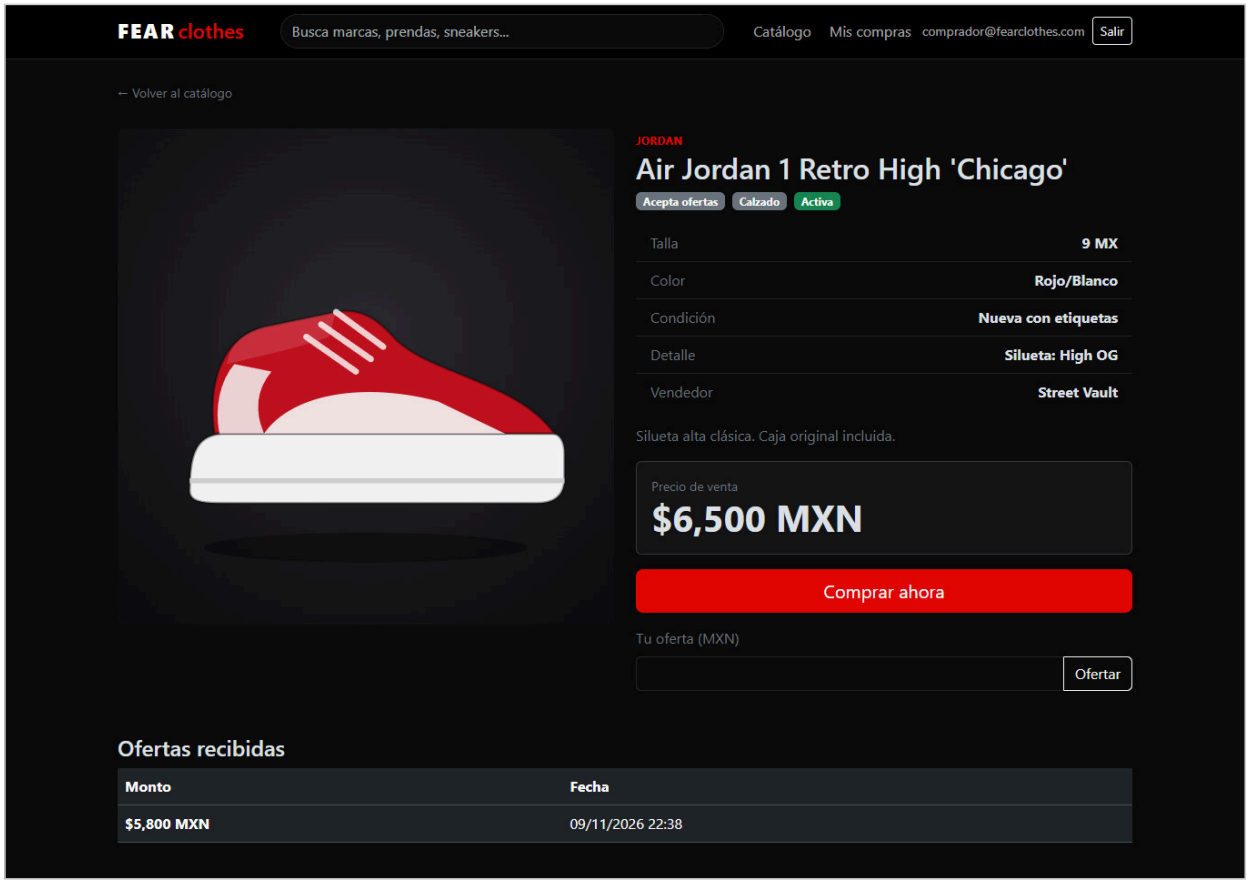


Figura 9. Detalle con ofertas (vista del comprador). Botón «Comprar ahora» y formulario de oferta.

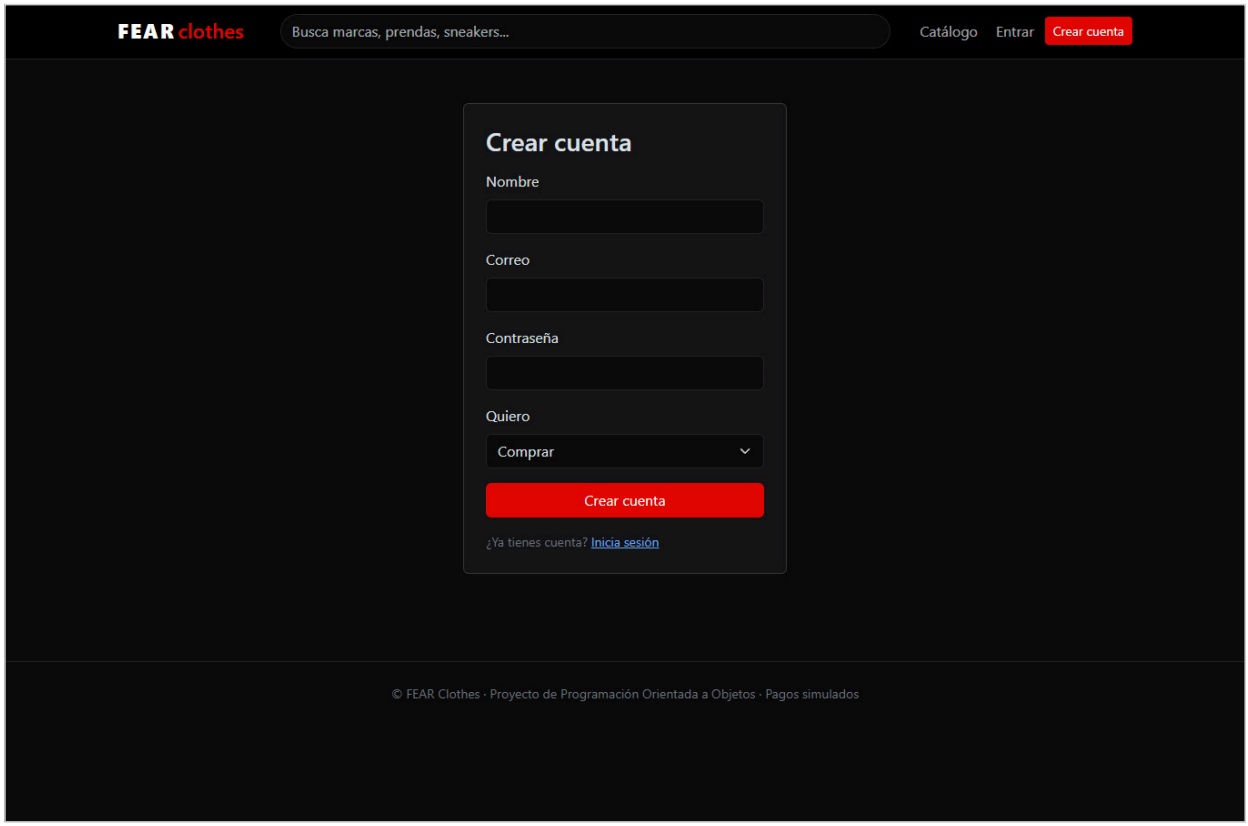


Figura 10. Registro de usuario. Formulario con validaciones.

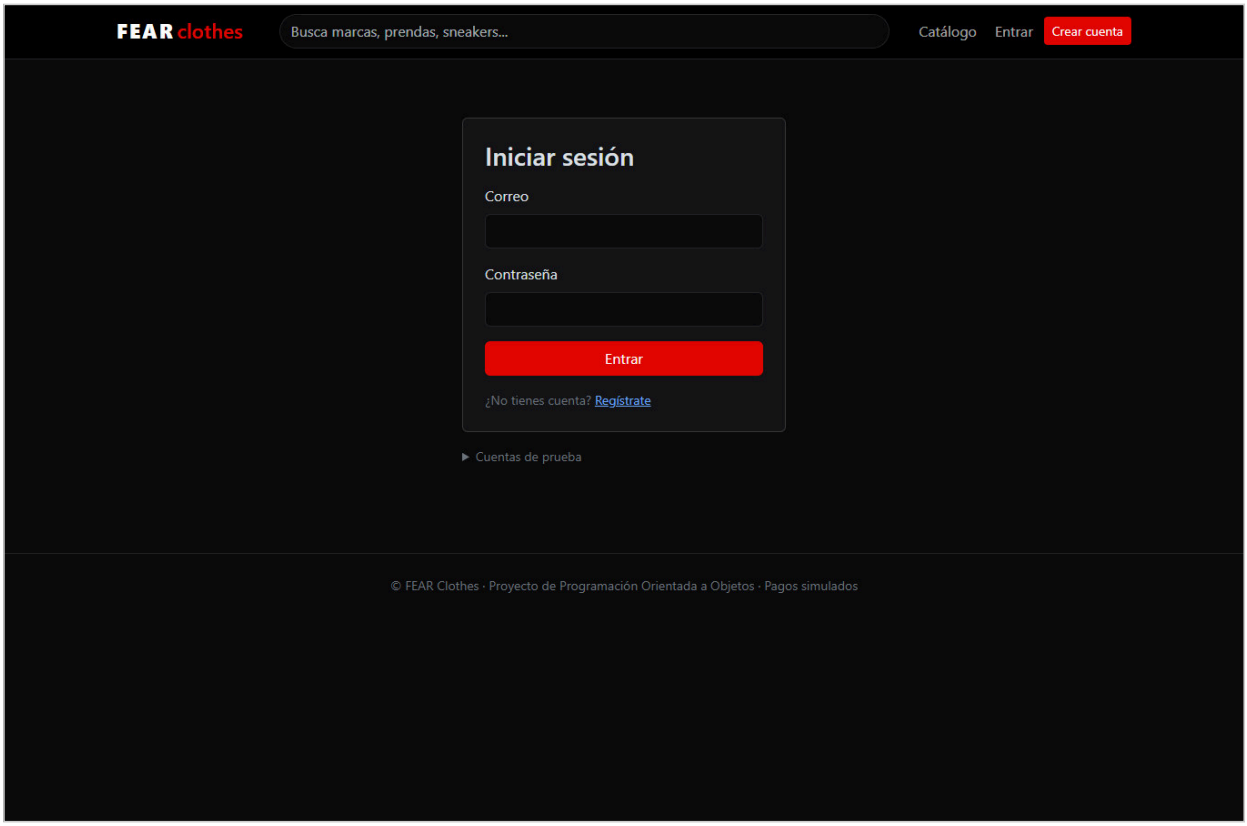


Figura 11. Inicio de sesión. Formulario y cuentas de prueba.

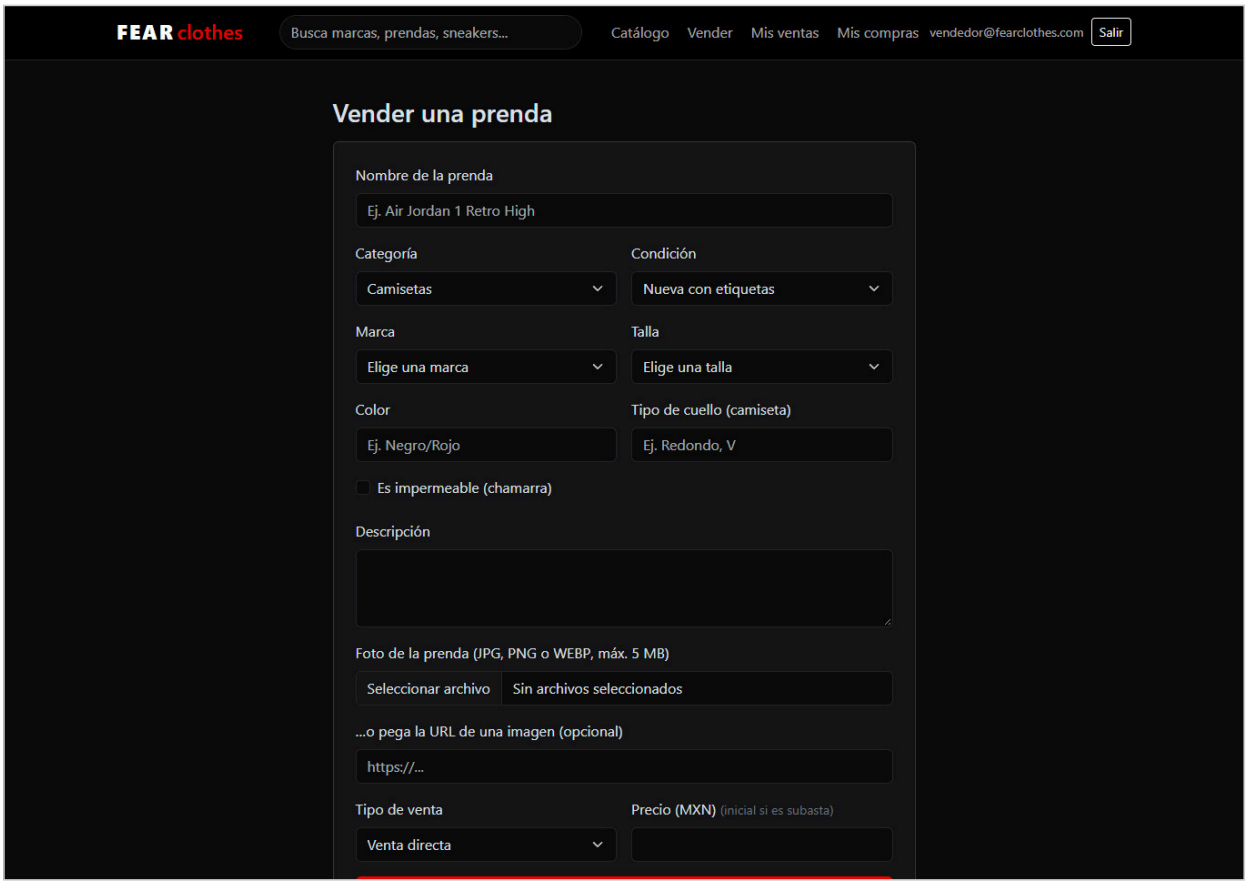


Figura 12. Publicar una prenda. Formulario con subida de imagen y vista previa.

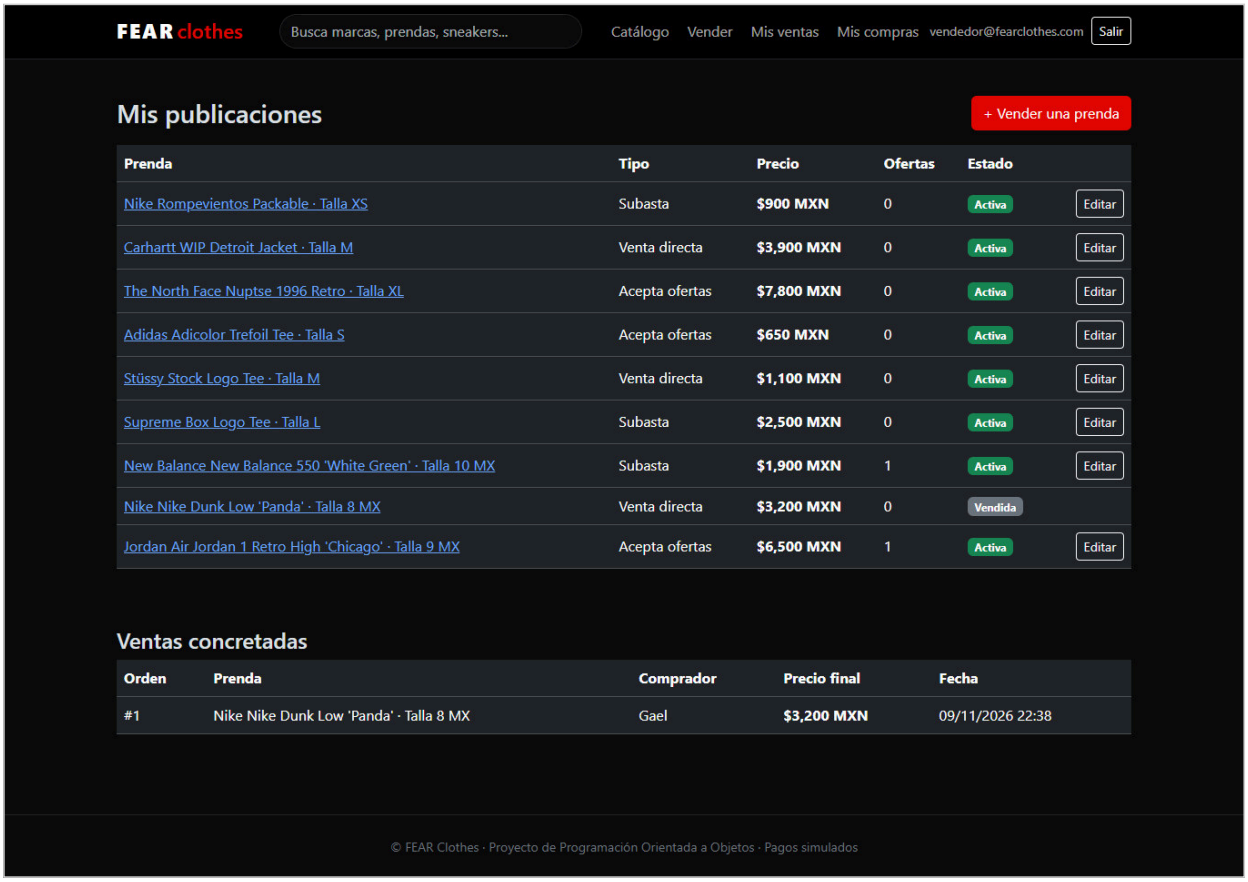


Figura 13. Mis ventas. Publicaciones propias y ventas concretadas.

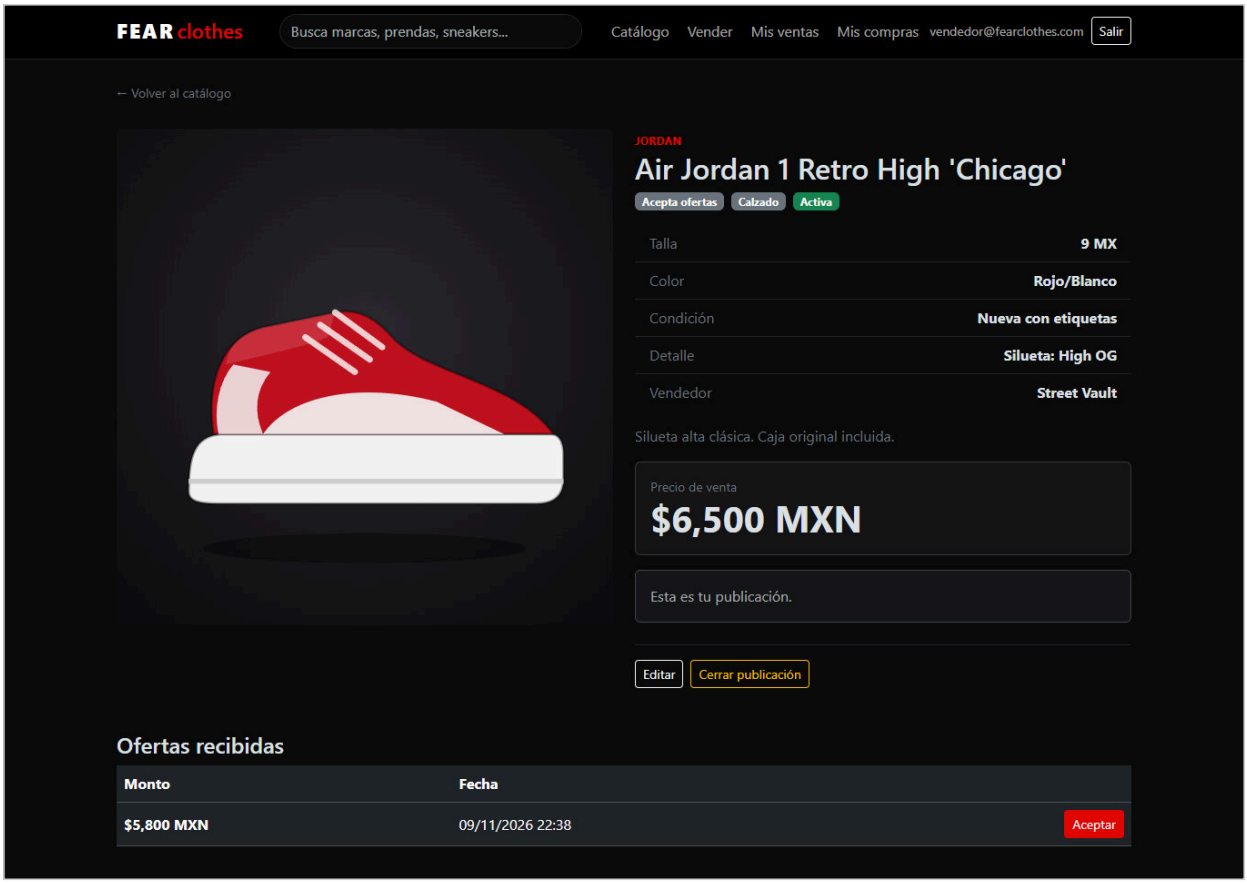


Figura 14. Aceptar una oferta. Tabla de ofertas con el botón Aceptar.

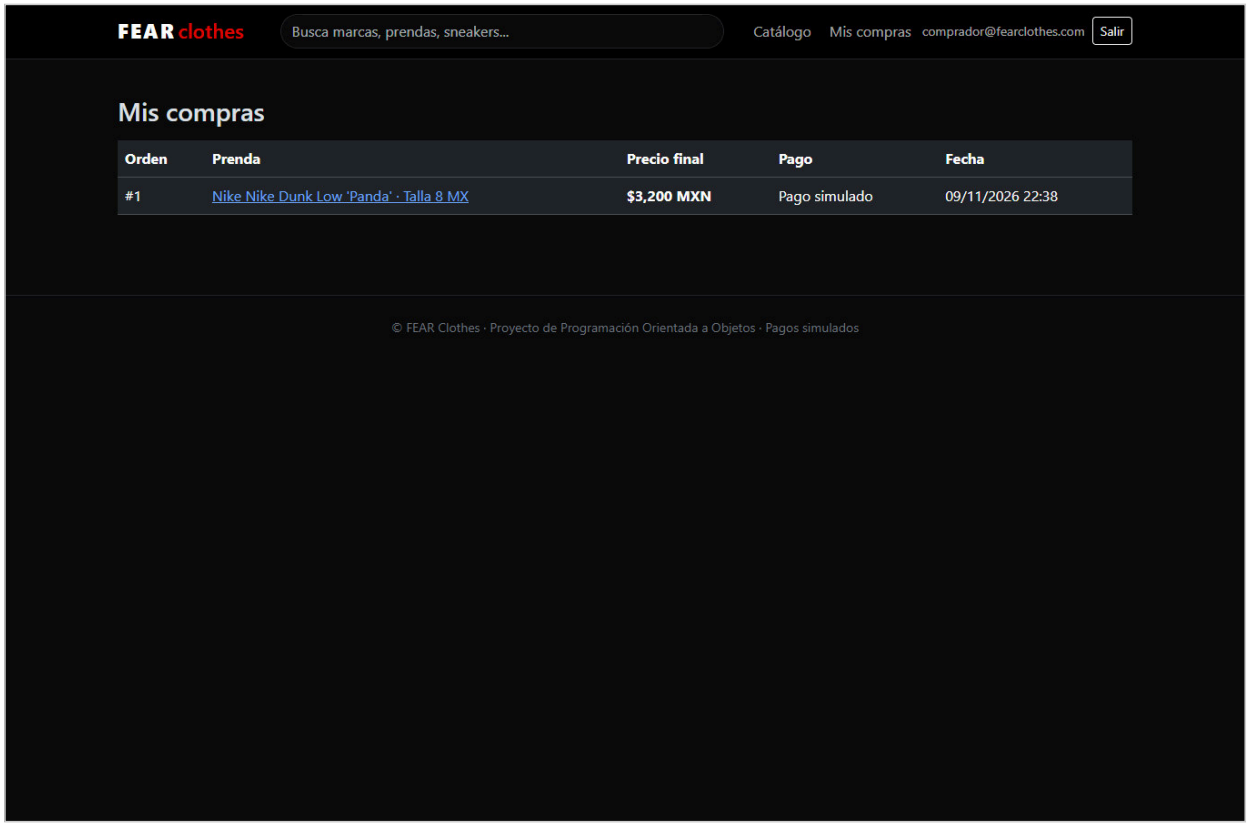


Figura 15. Mis compras. Historial de órdenes del comprador.

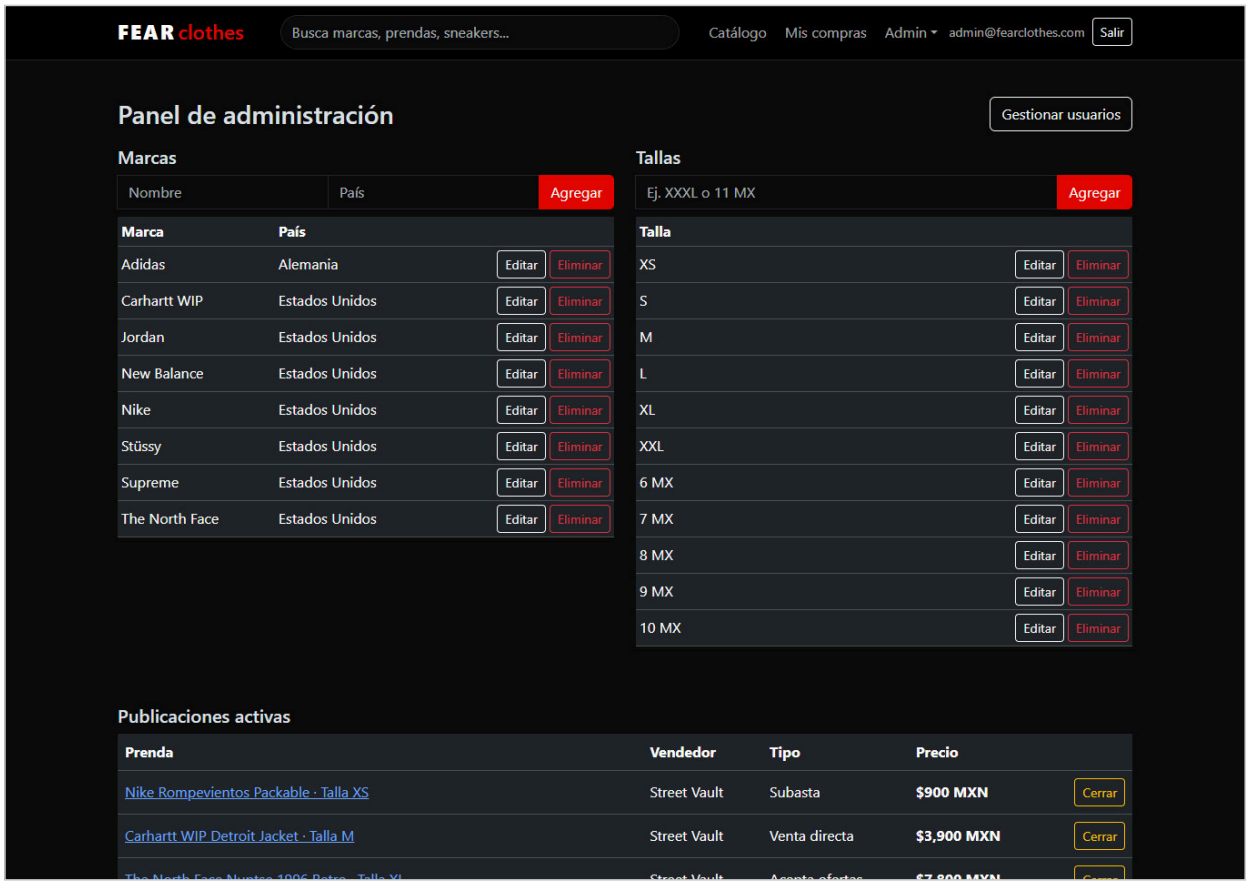


Figura 16. Panel de administración. Marcas, tallas y publicaciones activas.

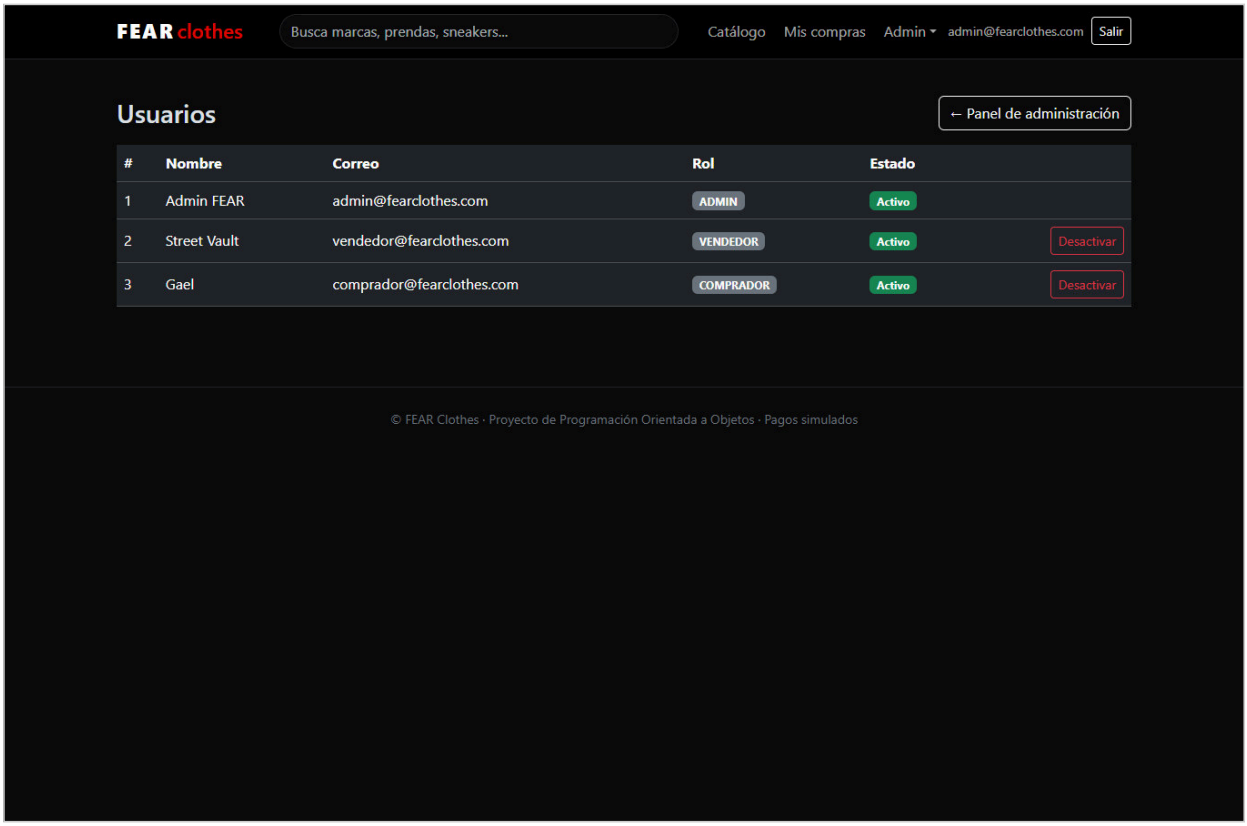


Figura 17. Administración de usuarios. Listado de cuentas con activar/desactivar.

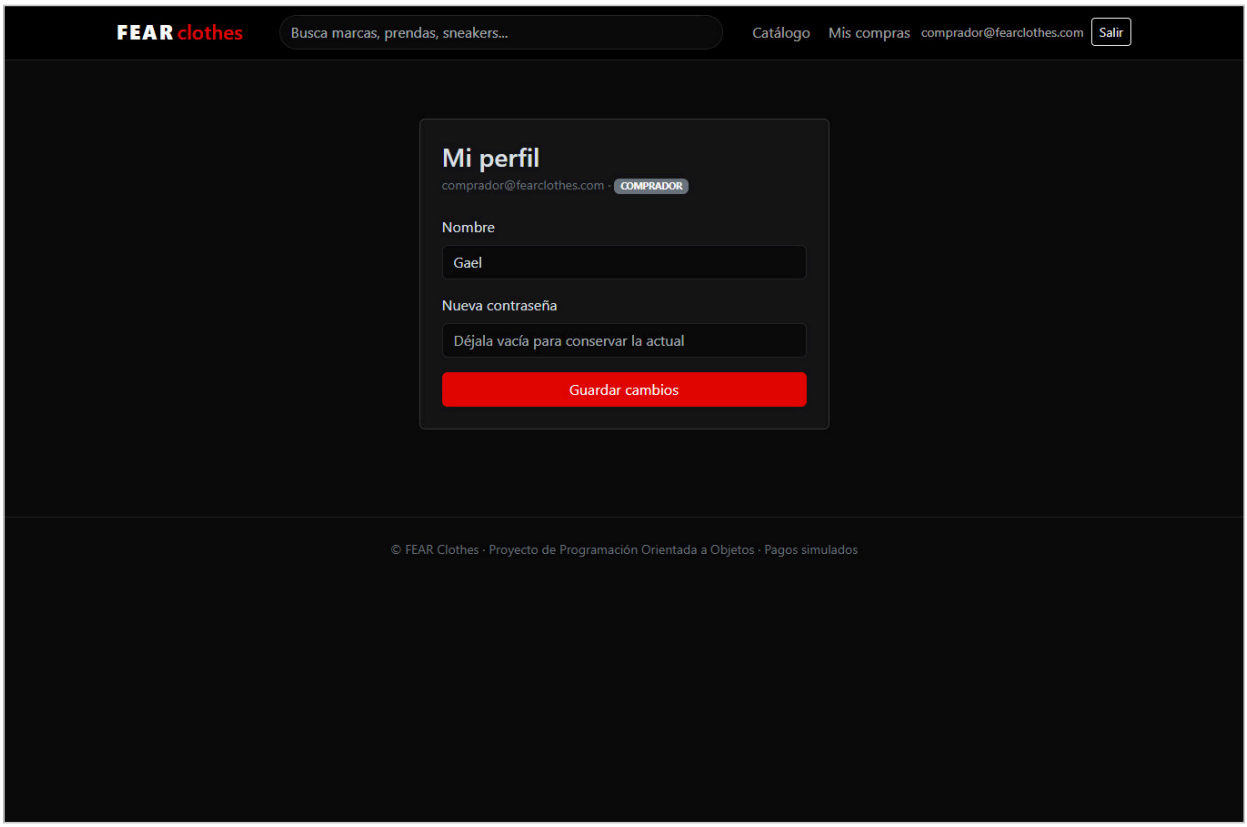


Figura 18. Mi perfil. Edición de nombre y contraseña.

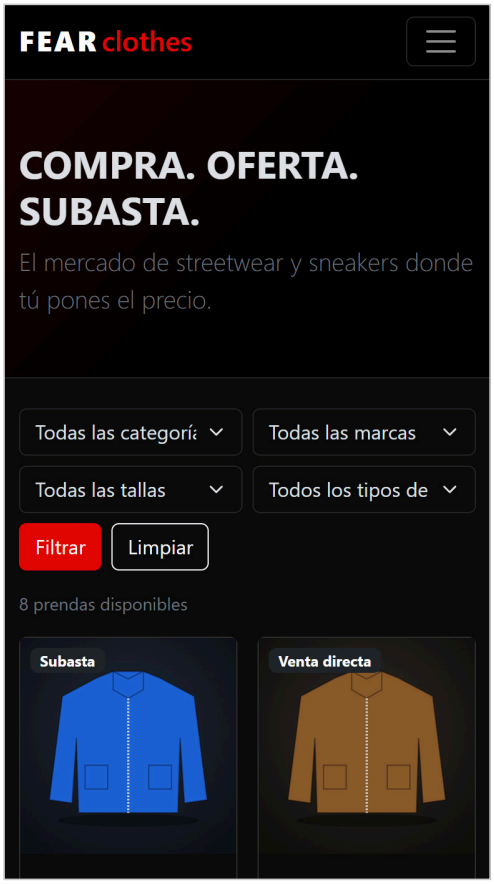


Figura 19. Vista en teléfono. Diseño responsivo con Bootstrap.

16. Conclusiones

El proyecto permitió construir una aplicación web completa a partir de un problema con reglas de negocio reales, integrando los conceptos de la Programación Orientada a Objetos en una arquitectura MVC por capas.

El patrón **Strategy**, apoyado en una clase base con **Template Method**, resultó la decisión de diseño más valiosa: agregar un nuevo tipo de venta equivale a escribir una clase que implemente `TipoVenta` y registrarla en la fábrica, sin modificar controladores ni servicios. De igual forma, las jerarquías de `Usuario` y `Prenda` mostraron cómo la herencia y el polimorfismo reducen condicionales y concentran el comportamiento en las clases correctas.

En el modelado de datos, trasladar la herencia a tablas obligó a evaluar alternativas y a justificar la normalización: la estrategia de tabla por subclase evita columnas vacías y mantiene la base en tercera forma normal. El uso de JPA/Hibernate y de repositorios permitió que el código fuera independiente del motor, con H2 en desarrollo y MySQL previsto para AWS.

Como aprendizajes, el equipo reforzó la separación de responsabilidades, la validación en el servidor, el manejo de seguridad con roles y la importancia de documentar con diagramas. Quedan como siguientes pasos el despliegue en la nube, la automatización de pruebas y la integración de pagos reales.

17. Referencias Bibliográficas

Bloch, J. (2018). *Effective Java* (3.ª ed.). Addison-Wesley.

Elmasri, R., y Navathe, S. B. (2016). *Fundamentals of Database Systems* (7.ª ed.). Pearson.

Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.

Fowler, M. (2004). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3.ª ed.). Addison-Wesley.

Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

Sommerville, I. (2016). *Software Engineering* (10.ª ed.). Pearson.

Bootstrap. (s. f.). *Bootstrap documentation*. https://getbootstrap.com/docs/

DBML. (s. f.). *DBML documentation*. https://dbml.dbdiagram.io/docs/

Hibernate. (s. f.). *Hibernate ORM documentation*. https://hibernate.org/orm/documentation/

Oracle. (s. f.). *Java documentation*. https://docs.oracle.com/en/java/

Spring. (s. f.). *Spring Boot reference documentation*. https://docs.spring.io/spring-boot/

Spring. (s. f.). *Spring Data JPA reference documentation*. https://docs.spring.io/spring-data/jpa/reference/

Spring. (s. f.). *Spring Security reference documentation*. https://docs.spring.io/spring-security/reference/

Thymeleaf. (s. f.). *Thymeleaf documentation*. https://www.thymeleaf.org/documentation.html

Amazon Web Services. (s. f.). *AWS documentation: Amazon EC2, Amazon RDS y Amazon S3*. https://docs.aws.amazon.com/